

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG HỆ THỐNG CHẤM THI TỰ ĐỘNG
BẰNG OMR HỖ TRỢ ĐA NỀN TẢNG

SMART GRADING SYSTEM

Sinh viên thực hiện : [TÊN SINH VIÊN]

Mã số sinh viên : [MSSV]

Lớp : [LỚP]

Giáo viên hướng dẫn : [TÊN GVHD]

Hà Nội, 06/2026

Mục lục

1	CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI	11
1.1	Đặt vấn đề	11
1.2	Mục tiêu và phạm vi đề tài	12
1.2.1	Tổng quan các nghiên cứu và sản phẩm hiện có	12
1.2.2	Hạn chế của các giải pháp hiện có	13
1.2.3	Mục tiêu của đề tài	13
1.2.4	Phạm vi đề tài	13
1.3	Định hướng giải pháp	14
1.4	Bố cục đồ án	15
2	CHƯƠNG 2: KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU	17
2.1	Khảo sát hiện trạng	17
2.1.1	Khảo sát thực trạng chấm thi trắc nghiệm	17
2.1.2	Khảo sát các giải pháp hiện có	18
2.1.3	Nhận xét và đánh giá	19
2.2	Tổng quan chức năng	19
2.2.1	Các tác nhân hệ thống	19
2.2.2	Biểu đồ Use Case tổng quát	20
2.2.3	Mô hình hóa quy trình nghiệp vụ	20
2.3	Đặc tả chức năng	22
2.3.1	Đặc tả Use Case Quản lý đề thi	22
2.3.2	Đặc tả Use Case Quét phiếu trả lời	23
2.3.3	Đặc tả Use Case Xem kết quả thi	25
2.4	Yêu cầu phi chức năng	26
2.4.1	Yêu cầu về hiệu năng	26
2.4.2	Yêu cầu về bảo mật	26
2.4.3	Yêu cầu về tính khả dụng	27
2.4.4	Yêu cầu về tính dễ bảo trì	27
3	CHƯƠNG 3: NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	29
3.1	Công nghệ nhận diện đánh dấu quang học OMR	29
3.1.1	Giới thiệu về OMR	29
3.1.2	Quy trình xử lý ảnh OMR	30
3.1.3	Các kỹ thuật xử lý ảnh nâng cao	31
3.2	Framework Flutter cho ứng dụng di động	32
3.2.1	Giới thiệu về Flutter	32
3.2.2	Kiến trúc ứng dụng Flutter	32
3.2.3	State Management với flutter_bloc	33

3.3	React và TypeScript cho ứng dụng web	34
3.3.1	Giới thiệu về React	34
3.3.2	Kiến trúc ứng dụng React	35
3.3.3	State Management với Zustand	36
3.4	Node.js và MongoDB cho Backend	37
3.4.1	Giới thiệu về Node.js	37
3.4.2	Giới thiệu về MongoDB	37
3.4.3	Kiến trúc Backend	39
3.5	Lựa chọn và so sánh công nghệ	40
4	CHƯƠNG 4: PHÂN TÍCH THIẾT KẾ VÀ TRIỂN KHAI	43
4.1	Thiết kế kiến trúc	43
4.1.1	Lựa chọn kiến trúc phần mềm	43
4.1.2	Kiến trúc Mobile App (Flutter)	44
4.1.3	Kiến trúc Web App (React)	45
4.1.4	Kiến trúc Backend (Node.js)	46
4.2	Thiết kế cơ sở dữ liệu	47
4.2.1	Sơ đồ ERD	47
4.2.2	Thiết kế Collection	48
4.3	Thiết kế chi tiết các thành phần	49
4.3.1	OMR Engine Design	49
4.3.2	Sequence Diagram cho chức năng quét phiếu	50
4.3.3	Giao diện người dùng	51
4.4	Xây dựng ứng dụng	52
4.4.1	Công cụ phát triển	52
4.4.2	Triển khai OMR Engine	53
4.4.3	Triển khai API Endpoints	54
4.5	Kiểm thử	55
4.5.1	Kiểm thử OMR Engine	55
4.5.2	Kiểm thử API	56
4.6	Triển khai thực tế	56
4.6.1	Mô hình triển khai	56
4.6.2	Cấu hình môi trường	57
5	CHƯƠNG 5: CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	59
5.1	Giải pháp 1: Thuật toán OMR thích ứng đa điều kiện	59
5.1.1	Bài toán đặt ra	59
5.1.2	Giải pháp đề xuất	59
5.1.3	Chi tiết kỹ thuật	60
5.1.4	Kết quả đạt được	62
5.2	Giải pháp 2: Kiến trúc Offline-First cho ứng dụng di động	63
5.2.1	Bài toán đặt ra	63
5.2.2	Giải pháp đề xuất	63
5.2.3	Chi tiết kỹ thuật	64
5.2.4	Kết quả đạt được	66
5.3	Giải pháp 3: Tối ưu hóa hiệu năng OMR trên thiết bị di động	66
5.3.1	Bài toán đặt ra	66
5.3.2	Giải pháp đề xuất	67

5.3.3	Kết quả đạt được	69
5.4	Giải pháp 4: Tích hợp AMC cho sinh đề thi đa phiên bản	69
5.4.1	Bài toán đặt ra	69
5.4.2	Giải pháp đề xuất	70
5.5	Tổng hợp đóng góp của đề tài	71
6	CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	73
6.1	Kết luận	73
6.1.1	Tổng kết kết quả đạt được	73
6.1.2	So sánh với các giải pháp hiện có	74
6.1.3	Bài học kinh nghiệm	75
6.2	Hạn chế và khuyết điểm	75
6.3	Hướng phát triển	75
6.3.1	Hoàn thiện tính năng hiện có	76
6.3.2	Mở rộng tính năng mới	76
6.3.3	Nghiên cứu và cải tiến kỹ thuật	76
A	HƯỚNG DẪN CÀI ĐẶT VÀ SỬ DỤNG	79
A.1	Cài đặt môi trường phát triển	79
A.1.1	Yêu cầu hệ thống	79
A.1.2	Cài đặt Backend	79
A.1.3	Cài đặt Mobile App	79
A.1.4	Cài đặt Web App	80
A.2	Hướng dẫn sử dụng	80
A.2.1	Đối với giáo viên	80
A.2.2	Đối với học sinh	80
A.3	Xử lý sự cố	81
A.3.1	Lỗi kết nối database	81
A.3.2	Lỗi build Flutter	81
A.3.3	Lỗi CORS	81

Danh sách hình vẽ

2.1	Biểu đồ Use Case tổng quan hệ thống SMART GRADING	20
2.2	Biểu đồ Sequence cho quy trình quét phiếu OMR	21
2.3	Biểu đồ Use Case phân rã cho chức năng quét phiếu OMR	25
3.1	Luồng xử lý OMR từ thu ảnh đến trích xuất kết quả	30
3.2	Kiến trúc Clean Architecture trong ứng dụng Flutter	33
3.3	Cấu trúc thư mục ứng dụng React	36
3.4	Kiến trúc backend Node.js theo phân tầng	40
4.1	Kiến trúc tổng quan hệ thống SMART GRADING	44
4.2	Sơ đồ ERD cho hệ thống SMART GRADING	48
4.3	Biểu đồ Sequence cho quy trình quét phiếu OMR	51
4.4	Mockup giao diện ứng dụng Mobile	52
4.5	Mô hình triển khai hệ thống SMART GRADING	57
5.1	Luồng thuật toán AMCOMR	60
5.2	Kiến trúc Offline-First với Smart Sync	64
5.3	Luồng tích hợp AMC Bridge Service	70

Danh sách bảng

1.1	So sánh các giải pháp OMR hiện có	13
2.1	So sánh chi tiết các giải pháp chấm thi trắc nghiệm	18
2.2	Đặc tả Use Case UC01: Quản lý đề thi	22
2.3	Đặc tả Use Case UC02: Quét phiếu trả lời	23
2.4	Đặc tả Use Case UC03: Xem kết quả thi	25
2.5	Tổng hợp yêu cầu phi chức năng của hệ thống	27
3.1	Các kỹ thuật xử lý ảnh OMR và ứng dụng	32
3.2	So sánh các lựa chọn công nghệ	40
4.1	Danh sách công cụ phát triển	53
4.2	Các API endpoint chính	54
4.3	Kết quả kiểm thử OMR Engine	55
4.4	Kết quả kiểm thử API	56
5.1	So sánh độ chính xác giữa Fixed Threshold và AMCOMR	62
5.2	So sánh hiệu năng trước và sau tối ưu hóa	69
6.1	So sánh kết quả đạt được với mục tiêu đề ra	74
6.2	So sánh SMART GRADING với các giải pháp hiện có	74

Chương 1

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

Tổng quan chương

Chương này trình bày bối cảnh và tính cấp thiết của đề tài, phân tích các vấn đề tồn tại trong thực tiễn chấm thi trắc nghiệm hiện nay. Trên cơ sở đó, chương đề xuất mục tiêu và phạm vi của đề tài, đồng thời giới thiệu tổng quan giải pháp được lựa chọn cùng các công nghệ sử dụng. Phần cuối chương trình bày bố cục của toàn bộ đề án.

1.1 Đặt vấn đề

Hiện nay, hình thức thi trắc nghiệm được sử dụng rộng rãi trong các cơ sở giáo dục tại Việt Nam, từ bậc phổ thông đến đại học, nhờ khả năng đánh giá nhanh chóng, khách quan và có thể áp dụng cho số lượng lớn thí sinh cùng lúc. Với sự phát triển của giáo dục, mỗi năm hàng triệu bài thi trắc nghiệm được tổ chức tại các trường học trên cả nước, từ các bài kiểm tra định kỳ, thi học kỳ đến các kỳ thi quan trọng như thi tốt nghiệp THPT. Tuy nhiên, quy trình chấm thi trắc nghiệm truyền thống vẫn đang đối mặt với nhiều thách thức nghiêm trọng.

Thực trạng chấm thi trắc nghiệm hiện nay tại Việt Nam cho thấy một bức tranh đáng lo ngại. Theo khảo sát thực tế, trung bình một giáo viên có thể chấm thủ công khoảng 100 bài thi trắc nghiệm mỗi giờ, điều này có nghĩa rằng với một lớp học 40 học sinh, việc chấm thi hoàn tất có thể mất từ 2 đến 3 ngày làm việc. Đối với các kỳ thi quy mô lớn như thi học kỳ toàn trường hay thi thử, số lượng bài thi có thể lên đến hàng nghìn, đòi hỏi nhiều giáo viên tham gia chấm thi trong thời gian dài. Quá trình chấm thi thủ công này không chỉ tốn kém về mặt thời gian mà còn gây ra những hệ lụy nghiêm trọng khác.

Sai sót trong quá trình chấm thi thủ công là một vấn đề không thể bỏ qua. Nghiên cứu trong lĩnh vực giáo dục đã chỉ ra rằng tỷ lệ sai sót khi chấm thi trắc nghiệm bằng tay dao động từ 0,5% đến 2%, tùy thuộc vào độ phức tạp của đề thi và tình trạng mệt mỏi của người chấm. Những sai sót này bao gồm việc đối chiếu nhầm đáp án, đọc sai số câu, tính nhầm điểm hoặc đánh dấu nhầm kết quả vào bảng điểm. Hậu quả của những sai sót này không chỉ ảnh hưởng đến điểm số của từng học sinh mà còn tác động đến sự công bằng trong thi cử và uy tín của nhà trường.

Một vấn đề quan trọng khác mà phương pháp chấm thi truyền thống không thể giải quyết là việc phát hiện các hành vi gian lận trong quá trình làm bài. Khi giáo viên đối chiếu đáp án bằng mắt thường, rất khó để nhận biết các trường hợp tô đúp

(đánh dấu hai đáp án cho cùng một câu hỏi), tô mờ hoặc tô lem ra ngoài vùng bubble. Những trường hợp này có thể dẫn đến tranh chấp điểm số giữa học sinh và giáo viên mà không có căn cứ khách quan để phân xử. Ngoài ra, chi phí thuê nhân viên chấm thi cho các kỳ thi quy mô lớn cũng là một gánh nặng tài chính đáng kể đối với các cơ sở giáo dục.

Từ những phân tích trên, bài toán đặt ra là cần xây dựng một hệ thống có khả năng tự động hóa hoàn toàn quy trình chấm thi trắc nghiệm, từ việc thu nhận phiếu trả lời đến tính toán và công bố kết quả. Hệ thống này phải đảm bảo độ chính xác cao, thời gian xử lý nhanh, có khả năng phát hiện các trường hợp bất thường như tô đúp hay tô mờ, đồng thời có thể triển khai dễ dàng với chi phí hợp lý. Đặc biệt, hệ thống cần hỗ trợ đa nền tảng để có thể sử dụng trên nhiều loại thiết bị khác nhau, từ điện thoại thông minh đến máy tính bảng và máy tính để bàn.

1.2 Mục tiêu và phạm vi đề tài

1.2.1 Tổng quan các nghiên cứu và sản phẩm hiện có

Trên thị trường hiện nay đã có nhiều giải pháp liên quan đến việc chấm thi trắc nghiệm, từ các công cụ miễn phí đến các hệ thống thương mại chuyên nghiệp. Việc khảo sát và đánh giá các giải pháp này giúp xác định rõ hơn những ưu điểm cần kế thừa cũng như những hạn chế cần khắc phục trong đề tài.

OMRChecker là một công cụ mã nguồn mở được phát triển bằng Python và OpenCV, cho phép nhận diện và chấm thi trắc nghiệm từ ảnh chụp phiếu trả lời. Ưu điểm nổi bật của OMRChecker là hoàn toàn miễn phí, mã nguồn mở để bất kỳ ai cũng có thể sử dụng và cải tiến. Tuy nhiên, công cụ này có giao diện dòng lệnh (CLI), đòi hỏi người dùng phải có kiến thức kỹ thuật nhất định để cấu hình và sử dụng. Ngoài ra, OMRChecker không có ứng dụng di động và không hỗ trợ tích hợp trực tiếp với các hệ thống quản lý học tập (LMS) hiện có.

Các giải pháp thương mại như Scantron của Mỹ cung cấp máy quét chuyên dụng kết hợp với phần mềm quản lý, cho độ chính xác rất cao (98-99%) và tốc độ xử lý nhanh. Tuy nhiên, chi phí đầu tư cho hệ thống Scantron rất lớn, từ 2.000 đến 5.000 USD cho thiết bị phần cứng cùng với phí license phần mềm hàng năm. Điều này khiến giải pháp này không phù hợp với hầu hết các trường học tại Việt Nam, đặc biệt là các trường ở vùng sâu vùng xa.

Một số giải pháp dựa trên nền tảng web như Google Forms cho phép tổ chức thi trắc nghiệm online, nhưng yêu cầu học sinh phải có thiết bị (máy tính, điện thoại) và kết nối internet trong suốt quá trình thi. Điều này không phù hợp với thực tế thi cử truyền thống tại Việt Nam, nơi học sinh vẫn quen làm bài trên giấy và các trường chưa trang bị đủ thiết bị cho tất cả học sinh.

Bảng 1.1: So sánh các giải pháp OMR hiện có

Tiêu chí	OMRChecker	Scantron	Google Forms	Đề tài
Chi phí triển khai	Miễn phí	Rất cao	Trung bình	Miễn phí
Cross-platform	Có (CLI)	Không	Có	Có
Mobile app	Không	Không	Có	Có
Độ chính xác	95%	98%	100%	95%
Dễ sử dụng	Thấp	Cao	Cao	Cao
Mã nguồn mở	Có	Không	Không	Có
Hỗ trợ offline	Có	Có	Không	Có

1.2.2 Hạn chế của các giải pháp hiện có

Từ quá trình khảo sát trên, có thể nhận thấy một số hạn chế chung của các giải pháp hiện tại. Thứ nhất, phần lớn các giải pháp miễn phí có giao diện phức tạp, đòi hỏi kiến thức kỹ thuật để vận hành, trong khi các giải pháp dễ sử dụng lại có chi phí rất cao. Thứ hai, chưa có giải pháp nào được thiết kế đặc biệt cho thị trường Việt Nam với đặc thù sử dụng tiếng Việt và theo chương trình giáo dục trong nước. Thứ ba, thiếu sự kết hợp giữa một ứng dụng di động thân thiện với người dùng thông thường và một hệ thống quản lý web chuyên nghiệp trong cùng một giải pháp tổng thể.

1.2.3 Mục tiêu của đề tài

Dựa trên những hạn chế đã được phân tích, đề tài “Xây dựng hệ thống chấm thi tự động bằng OMR hỗ trợ đa nền tảng” đặt ra các mục tiêu cụ thể như sau.

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống hoàn chỉnh cho phép tự động hóa quy trình chấm thi trắc nghiệm, từ khâu thu nhận phiếu trả lời thông qua camera của thiết bị di động, xử lý nhận diện đáp án bằng công nghệ OMR, cho đến việc lưu trữ, quản lý và công bố kết quả thi một cách nhanh chóng và chính xác.

Các mục tiêu cụ thể bao gồm: phát triển engine xử lý ảnh OMR dựa trên thư viện OpenCV với độ chính xác nhận diện đạt từ 95% trở lên, cho phép xử lý các ảnh chụp từ smartphone với điều kiện ánh sáng và góc chụp khác nhau; xây dựng ứng dụng di động Flutter cho phép người dùng quét phiếu trả lời trắc nghiệm bằng camera, nhận diện tự động các vùng tô đáp án và hiển thị kết quả chấm ngay lập tức; phát triển ứng dụng web dựa trên React và TypeScript cung cấp giao diện quản lý đề thi, ngân hàng câu hỏi, xem kết quả và thống kê báo cáo cho giáo viên và quản trị viên; thiết kế backend RESTful API dựa trên Node.js và MongoDB đảm bảo khả năng mở rộng, bảo mật và tích hợp dễ dàng với các hệ thống khác; và triển khai hệ thống hỗ trợ nhiều loại đề thi với số lượng câu hỏi khác nhau (15, 30, 50 câu) cùng khả năng sinh nhiều mã đề tự động.

1.2.4 Phạm vi đề tài

Phạm vi nghiên cứu và triển khai của đề tài tập trung vào các cơ sở giáo dục phổ thông và trung học tại Việt Nam, bao gồm các trường tiểu học, trung học cơ sở và trung học phổ thông, các trung tâm đào tạo và luyện thi. Về mặt kỹ thuật, đề tài giới hạn trong việc xử lý các câu hỏi trắc nghiệm dạng lựa chọn (single choice và multiple choice), chưa bao gồm nhận diện chữ viết tay hay câu hỏi tự luận. Quy mô hỗ trợ của hệ thống

được thiết kế đáp ứng tốt cho các lớp học có quy mô đến 1.000 học sinh, phù hợp với điều kiện thực tế của các trường học tại Việt Nam.

1.3 Định hướng giải pháp

Để đạt được các mục tiêu đề ra, đề tài lựa chọn và kết hợp nhiều công nghệ hiện đại, phù hợp với đặc thù của bài toán và khả năng triển khai thực tế.

Về công nghệ xử lý ảnh, hệ thống sử dụng thư viện OpenCV (Open Computer Vision) phiên bản 4.x kết hợp với ngôn ngữ lập trình Python để xây dựng engine nhận diện OMR. OpenCV là thư viện mã nguồn mở được sử dụng rộng rãi trong lĩnh vực thị giác máy tính, cung cấp hơn 2.500 thuật toán xử lý ảnh từ cơ bản đến chuyên sâu. Thư viện này có ưu điểm là hoàn toàn miễn phí, đa nền tảng (Windows, Linux, macOS, Android, iOS), được cộng đồng phát triển liên tục và có tài liệu phong phú. Ngoài ra, để tối ưu hiệu năng trên thiết bị di động, phần xử lý OMR cơ bản cũng được triển khai trực tiếp trên Flutter sử dụng các thư viện xử lý ảnh Dart.

Về nền tảng phát triển ứng dụng di động, đề tài sử dụng Flutter, framework cross-platform do Google phát triển. Flutter cho phép xây dựng ứng dụng native cho cả iOS và Android từ một codebase duy nhất, đảm bảo hiệu năng gần như native nhờ sử dụng engine đồ họa Skia. Các ưu điểm khác của Flutter bao gồm tính năng Hot Reload cho phép xem thay đổi ngay lập tức trong quá trình phát triển, hệ thống widget phong phú và có thể tùy chỉnh cao, cộng đồng phát triển đông đảo với nhiều thư viện hỗ trợ. Ngôn ngữ lập trình được sử dụng là Dart, cũng do Google phát triển, với cú pháp dễ học và hỗ trợ tốt cho lập trình hướng đối tượng.

Về nền tảng phát triển ứng dụng web, React kết hợp với TypeScript được lựa chọn để xây dựng giao diện quản lý. React là thư viện JavaScript được phát triển bởi Facebook, cho phép xây dựng giao diện người dùng theo kiểu component-based với khả năng tái sử dụng cao. TypeScript, một superset của JavaScript, mang lại lợi ích về kiểm tra kiểu tĩnh ở thời điểm biên dịch, giúp giảm thiểu lỗi và dễ dàng bảo trì mã nguồn lớn. Hệ sinh thái của React rất phong phú với nhiều thư viện hỗ trợ cho state management, routing, và data fetching.

Về backend, Node.js với framework Express và cơ sở dữ liệu MongoDB được sử dụng để xây dựng RESTful API. Node.js cho phép chạy JavaScript ở phía server, tận dụng ưu điểm của JavaScript trên toàn bộ stack phát triển. Express cung cấp một framework nhẹ và linh hoạt cho việc xây dựng các API web. MongoDB, một cơ sở dữ liệu NoSQL document-oriented, đặc biệt phù hợp với dữ liệu có cấu trúc linh hoạt như đề thi, câu hỏi và kết quả thi. Mongoose ODM được sử dụng để tương tác với MongoDB một cách type-safe và có cấu trúc.

Mô hình kiến trúc tổng thể của hệ thống gồm ba thành phần chính: ứng dụng di động Flutter cho phép người dùng quét phiếu trả lời trắc nghiệm và xem kết quả; ứng dụng web React cung cấp giao diện quản lý đề thi, câu hỏi và kết quả thi cho giáo viên và quản trị viên; và backend Node.js xử lý các yêu cầu từ cả hai ứng dụng client, quản lý dữ liệu trên MongoDB và thực hiện các nghiệp vụ phức tạp như sinh mã đề, tính điểm và tạo báo cáo.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án được tổ chức theo cấu trúc sau đây để trình bày một cách có hệ thống toàn bộ quá trình nghiên cứu, phân tích, thiết kế và triển khai hệ thống.

Chương 2 trình bày kết quả khảo sát hiện trạng về các giải pháp chấm thi trắc nghiệm hiện nay tại các cơ sở giáo dục, phân tích các sản phẩm tương tự trên thị trường để xác định ưu điểm và hạn chế của từng giải pháp. Chương này cũng trình bày chi tiết về yêu cầu chức năng và phi chức năng của hệ thống thông qua các biểu đồ use case, đặc tả các use case quan trọng và mô hình hóa quy trình nghiệp vụ.

Trong Chương 3, đề án giới thiệu các nền tảng lý thuyết và công nghệ được sử dụng trong quá trình triển khai, bao gồm OpenCV cho xử lý ảnh và nhận diện OMR, Flutter cho phát triển ứng dụng di động đa nền tảng, React và TypeScript cho xây dựng giao diện web, Node.js và Express cho backend API, cũng như MongoDB cho lưu trữ dữ liệu. Chương này giải thích nguyên lý hoạt động của công nghệ OMR và các kỹ thuật xử lý ảnh liên quan.

Chương 4 trình bày chi tiết về phân tích và thiết kế hệ thống theo phương pháp hướng đối tượng, bao gồm kiến trúc tổng thể, biểu đồ package và class, thiết kế cơ sở dữ liệu (ERD, schema), biểu đồ trình tự cho các use case chính, và thiết kế giao diện người dùng. Phần triển khai và kết quả thực nghiệm, bao gồm các công cụ sử dụng, thống kê mã nguồn và kết quả kiểm thử, cũng được trình bày trong chương này.

Chương 5 tập trung vào việc trình bày các giải pháp kỹ thuật nổi bật và đóng góp của đề án, đặc biệt là thuật toán nhận diện OMR với độ chính xác cao, kỹ thuật xử lý ảnh thích ứng với các điều kiện chụp khác nhau, giải pháp phát hiện tô đúp và tô mờ, cũng như kiến trúc offline-first cho ứng dụng di động. Các kết quả đo lường hiệu năng và so sánh với các giải pháp hiện có được trình bày để đánh giá hiệu quả của các giải pháp đề xuất.

Chương 6 là phần kết luận và hướng phát triển của đề tài. Chương này tổng kết những kết quả đạt được, so sánh với các mục tiêu đã đề ra ban đầu, phân tích những hạn chế và bài học kinh nghiệm rút ra trong quá trình thực hiện. Đồng thời, chương cũng đề xuất các hướng nghiên cứu và phát triển tiếp theo để hoàn thiện và mở rộng hệ thống trong tương lai.

Phần phụ lục bao gồm các tài liệu bổ sung như hướng dẫn cài đặt và sử dụng hệ thống, đặc tả chi tiết các use case, và các tài liệu tham khảo được sử dụng trong quá trình nghiên cứu.

Kết chương

Chương 1 đã trình bày bối cảnh và tính cấp thiết của đề tài, xác định rõ các vấn đề tồn tại trong thực tiễn chấm thi trắc nghiệm hiện nay như tốn thời gian, dễ sai sót và khó phát hiện gian lận. Qua khảo sát các giải pháp hiện có, đề tài xác định được những hạn chế chung và đề xuất hướng giải quyết phù hợp. Các mục tiêu cụ thể đã được đặt ra với độ đo lường rõ ràng, và phạm vi đề tài được xác định phù hợp với khả năng triển khai. Việc lựa chọn các công nghệ như OpenCV, Flutter, React, Node.js và MongoDB được lý giải chi tiết dựa trên ưu điểm của từng công nghệ và sự phù hợp với bài toán đặt ra. Chương tiếp theo sẽ trình bày chi tiết về khảo sát hiện trạng và phân tích yêu cầu của hệ thống.

Chương 2

CHƯƠNG 2: KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Tổng quan chương

Chương 1 đã trình bày bối cảnh, mục tiêu và giải pháp đề xuất cho đề tài. Chương này thực hiện khảo sát chi tiết hiện trạng về quy trình chấm thi trắc nghiệm, phân tích các sản phẩm tương tự trên thị trường để xác định rõ hơn các yêu cầu cần đáp ứng. Trên cơ sở đó, chương tiến hành đặc tả các yêu cầu chức năng và phi chức năng của hệ thống, bao gồm các biểu đồ use case, đặc tả use case chi tiết, mô hình hóa quy trình nghiệp vụ và các yêu cầu về hiệu năng, bảo mật, tính khả dụng.

2.1 Khảo sát hiện trạng

2.1.1 Khảo sát thực trạng chấm thi trắc nghiệm

Quy trình chấm thi trắc nghiệm truyền thống tại các trường học Việt Nam hiện nay bao gồm nhiều bước tuần tự và thường kéo dài trong nhiều ngày. Bước đầu tiên là giáo viên phát đề thi kèm theo phiếu trả lời cho học sinh. Sau khi thí sinh hoàn thành bài thi, giáo viên thu lại toàn bộ phiếu trả lời. Tiếp theo, giáo viên sử dụng đáp án để đối chiếu và đánh dấu từng câu trả lời, đếm số câu đúng và tính điểm. Cuối cùng, điểm số được ghi vào bảng điểm và công bố cho học sinh.

Quy trình này có những đặc điểm và hạn chế đáng chú ý. Về mặt tích cực, phương pháp chấm tay có tính linh hoạt cao, giáo viên có thể kiểm soát trực tiếp toàn bộ quá trình và dễ dàng phát hiện các trường hợp bất thường khi đối chiếu trực tiếp. Tuy nhiên, về mặt tiêu cực, quy trình này đòi hỏi nhiều thời gian và công sức, đặc biệt với các kỳ thi có quy mô lớn. Một giáo viên chấm trung bình khoảng 100 bài mỗi giờ, có nghĩa là một lớp 40 học sinh cần khoảng 2-3 giờ chấm thi, chưa kể thời gian kiểm tra lại và xử lý các trường hợp tranh chấp.

Nghiêm trọng hơn, quá trình chấm thi thủ công tiềm ẩn nhiều nguy cơ sai sót. Theo các nghiên cứu trong lĩnh vực giáo dục, tỷ lệ sai sót khi chấm thi trắc nghiệm bằng tay dao động từ 0,5% đến 2%. Các loại sai sót phổ biến bao gồm: đánh dấu nhầm đáp án do mệt mỏi khi chấm số lượng lớn bài liên tục; tính nhầm tổng số câu đúng do nhầm lẫn giữa các bài thi; ghi nhầm điểm vào bảng điểm; và đối chiếu sai đáp án do in ấn mờ hoặc không rõ ràng. Ngoài ra, việc phát hiện các trường hợp tô đúp (học sinh đánh

dấu hai đáp án cho cùng một câu) hoặc tô mờ bằng mắt thường là rất khó khăn, dẫn đến tranh chấp khi công bố điểm.

Một khía cạnh khác cần xem xét là chi phí vận hành. Đối với các kỳ thi quy mô lớn như thi học kỳ, thi thử đại học, trường thường phải huy động nhiều giáo viên tham gia chấm thi, chi phí nhân công có thể lên đến hàng chục triệu đồng cho một đợt thi. Chưa kể đến chi phí giấy in đáp án, phiếu trả lời và các vật tư khác.

2.1.2 Khảo sát các giải pháp hiện có

Trên thị trường hiện nay tồn tại nhiều giải pháp hỗ trợ chấm thi trắc nghiệm với mức độ phức tạp và chi phí khác nhau. Việc khảo sát chi tiết các giải pháp này giúp xác định rõ những ưu điểm cần kế thừa và những hạn chế cần khắc phục.

OMRChecker là một công cụ mã nguồn mở phổ biến được phát triển bằng Python và OpenCV. Công cụ này cho phép nhận diện và chấm thi trắc nghiệm từ ảnh chụp phiếu trả lời với độ chính xác khoảng 95%. OMRChecker có ưu điểm là hoàn toàn miễn phí, mã nguồn mở nên bất kỳ ai cũng có thể sử dụng và tùy chỉnh theo nhu cầu. Tuy nhiên, OMRChecker hoạt động qua giao diện dòng lệnh (Command Line Interface), đòi hỏi người dùng phải có kiến thức kỹ thuật nhất định về Python và xử lý ảnh để cấu hình, chạy và xử lý lỗi. Ngoài ra, OMRChecker không có ứng dụng di động tích hợp và không hỗ trợ tốt cho việc quản lý đề thi, ngân hàng câu hỏi.

Scantron là hệ thống thương mại đến từ Mỹ, bao gồm máy quét chuyên dụng và phần mềm quản lý. Hệ thống này cung cấp độ chính xác rất cao (98-99%) và tốc độ xử lý nhanh, có thể quét hàng nghìn phiếu trong vài phút. Tuy nhiên, chi phí đầu tư ban đầu rất lớn, từ 2.000 đến 5.000 USD cho thiết bị phần cứng cùng với phí license phần mềm hàng năm. Điều này khiến Scantron không phù hợp với hầu hết các trường học tại Việt Nam, đặc biệt là các trường ở vùng sâu vùng xa.

Google Forms và các nền tảng quiz online khác cho phép tổ chức thi trắc nghiệm trực tuyến với ưu điểm là dễ sử dụng và miễn phí cho mục đích giáo dục. Tuy nhiên, các giải pháp này yêu cầu học sinh phải có thiết bị (máy tính, điện thoại thông minh) và kết nối internet ổn định trong suốt quá trình thi. Điều này không phù hợp với thực tế thi cử truyền thống tại Việt Nam, nơi học sinh vẫn quen làm bài trên giấy và nhiều trường chưa trang bị đủ thiết bị cho tất cả học sinh.

Bảng 2.1: So sánh chi tiết các giải pháp chấm thi trắc nghiệm

Tiêu chí	OMRChecker	Scantron	Google Forms	SMART GRADING
Chi phí triển khai	Miễn phí	2.000-5.000 USD	Miễn phí-edu	Miễn phí
Phần cứng yêu cầu	Máy tính	Máy quét chuyên dụng	Thiết bị có internet	Smartphone
Độ chính xác	95%	98-99%	100%	95%
Thời gian setup	Cao	Rất cao	Thấp	Trung bình
Giao diện mobile	Không	Không	Có	Có
Quản lý đề thi	Không	Có	Có	Có
Báo cáo thống kê	Cơ bản	Nâng cao	Cơ bản	Nâng cao
Hỗ trợ tiếng Việt	Thủ công	Không	Không	Có

2.1.3 Nhận xét và đánh giá

Từ quá trình khảo sát trên, có thể rút ra một số nhận xét quan trọng. Thứ nhất, các giải pháp miễn phí như OMRChecker thường có giao diện phức tạp, khó sử dụng cho người không có kiến thức kỹ thuật. Thứ hai, các giải pháp thương mại tuy hoàn thiện nhưng chi phí quá cao, không phù hợp với ngân sách của hầu hết trường học Việt Nam. Thứ ba, chưa có giải pháp nào được thiết kế đặc biệt cho thị trường Việt Nam, thiếu sự hỗ trợ tiếng Việt và chưa phù hợp với chương trình giáo dục trong nước. Thứ tư, thiếu sự kết hợp giữa một ứng dụng di động thân thiện với người dùng thông thường và một hệ thống quản lý web chuyên nghiệp trong cùng một giải pháp tổng thể.

Từ những nhận xét trên, đề tài hướng đến việc xây dựng một giải pháp có các đặc điểm: chi phí triển khai thấp (miễn phí cho người dùng cuối); giao diện thân thiện, dễ sử dụng cho cả giáo viên và học sinh; hỗ trợ đa nền tảng (iOS, Android, Web); hoạt động offline được để đảm bảo tính sẵn sàng trong mọi điều kiện; và có độ chính xác cao trong việc nhận diện đáp án.

2.2 Tổng quan chức năng

2.2.1 Các tác nhân hệ thống

Hệ thống SMART GRADING tương tác với bốn nhóm tác nhân chính, mỗi nhóm có các quyền hạn và chức năng riêng biệt.

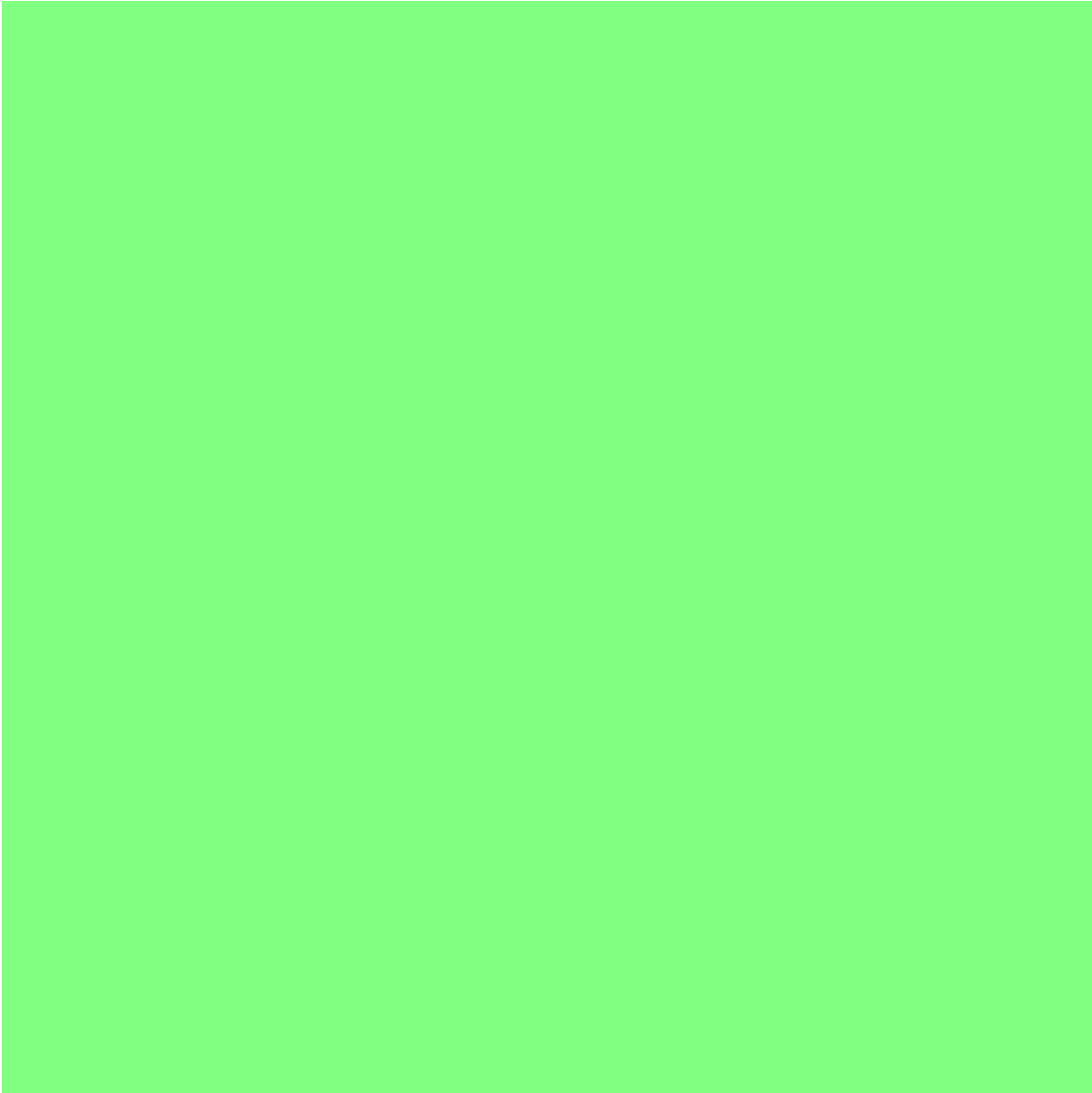
Giáo viên (Teacher) là nhóm người dùng chịu trách nhiệm quản lý các hoạt động liên quan đến bài thi. Giáo viên có thể tạo mới, chỉnh sửa và xóa đề thi theo nhu cầu của mình. Họ quản lý ngân hàng câu hỏi, bao gồm việc thêm câu hỏi mới, chỉnh sửa nội dung và đáp án, xóa câu hỏi không còn sử dụng. Giáo viên giao bài thi cho các lớp học, theo dõi tiến độ làm bài và xem kết quả chấm thi của học sinh. Ngoài ra, giáo viên còn có thể phúc khảo bài thi khi có khiếu nại từ học sinh.

Học sinh (Student) là nhóm người dùng chính của hệ thống, sử dụng ứng dụng để thực hiện các bài thi. Học sinh xem thông tin các bài thi đã được giao, bao gồm thời gian, thời lượng và số câu hỏi. Học sinh sử dụng camera của điện thoại để quét phiếu trả lời trắc nghiệm, nhận kết quả chấm ngay sau khi quét. Ngoài ra, học sinh có thể xem lịch sử các bài thi đã làm, điểm số chi tiết theo từng câu, và thực hiện khiếu nại điểm thi nếu phát hiện sai sót.

Quản trị viên (Admin) có quyền hạn cao nhất trong hệ thống. Quản trị viên quản lý toàn bộ người dùng, bao gồm việc thêm mới, chỉnh sửa thông tin và vô hiệu hóa tài khoản. Họ quản lý cấu hình hệ thống, thiết lập các thông số mặc định và quản lý quyền truy cập. Quản trị viên cấp trường có thể quản lý các lớp học, môn học và giáo viên trong trường của mình.

Hệ thống (System) đóng vai trò tự động hóa các quy trình trong hệ thống. Hệ thống tự động xử lý ảnh OMR, nhận diện đáp án và chấm điểm. Hệ thống sinh mã đề tự động dựa trên các tham số được cấu hình. Các thông báo được gửi tự động đến người dùng khi có cập nhật liên quan.

2.2.2 Biểu đồ Use Case tổng quát



Hình 2.1: Biểu đồ Use Case tổng quan hệ thống SMART GRADING

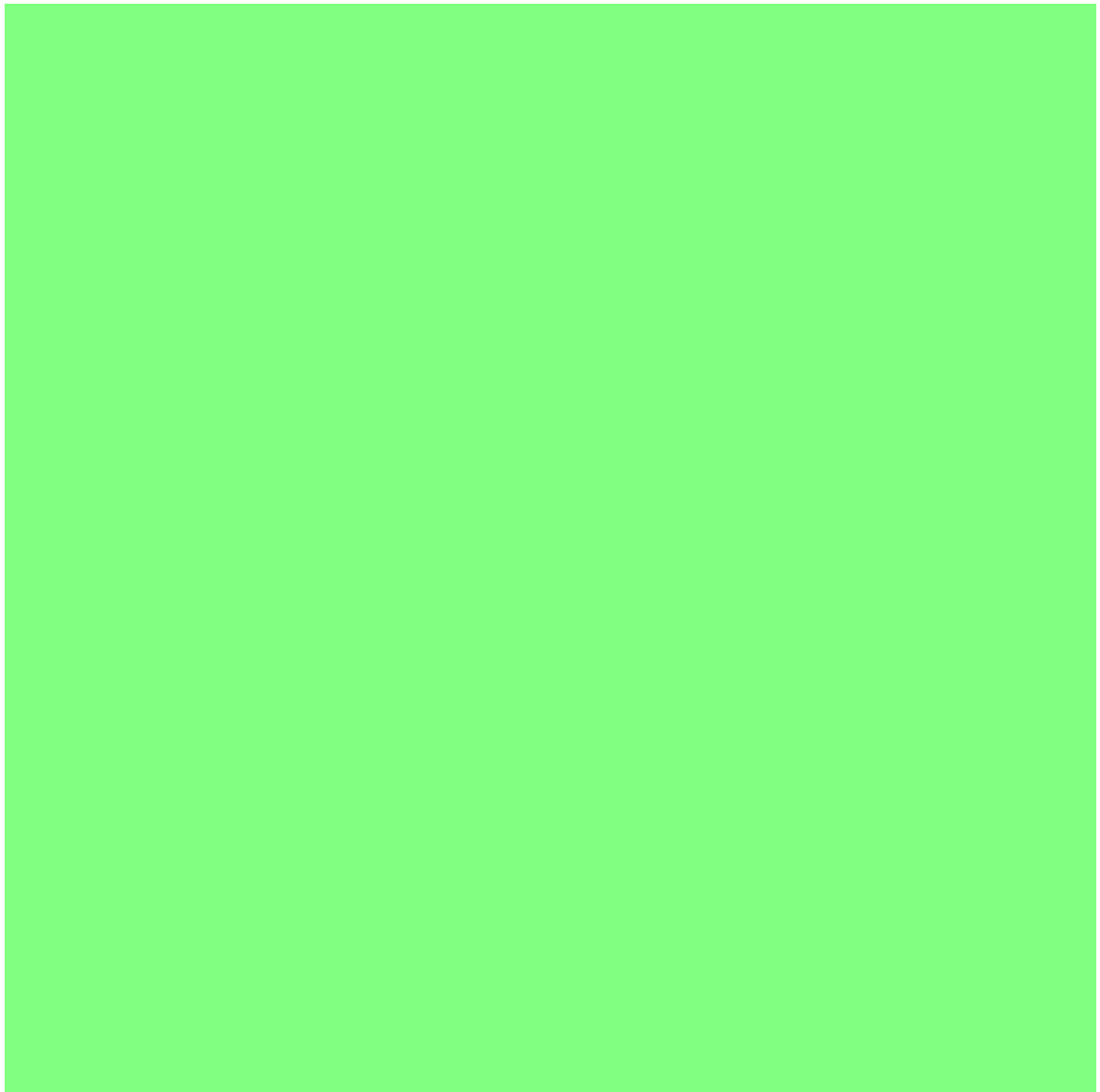
Biểu đồ Use Case tổng quan thể hiện các chức năng chính của hệ thống và mối quan hệ giữa các tác nhân với các chức năng đó. Các Use Case chính bao gồm: UC1 - Quản lý đề thi (bao gồm tạo, sửa, xóa, xem chi tiết đề thi); UC2 - Quản lý ngân hàng câu hỏi (bao gồm thêm, sửa, xóa, tìm kiếm câu hỏi); UC3 - Quét và nhận diện phiếu OMR; UC4 - Chấm điểm tự động; UC5 - Quản lý kết quả thi; UC6 - Phức khảo điểm; UC7 - Quản lý người dùng; UC8 - Thống kê báo cáo.

2.2.3 Mô hình hóa quy trình nghiệp vụ

Quy trình nghiệp vụ chính của hệ thống SMART GRADING bao gồm hai quy trình cốt lõi: quy trình tạo và quản lý đề thi, và quy trình quét phiếu và chấm thi.

Quy trình tạo và quản lý đề thi bắt đầu khi giáo viên đăng nhập vào hệ thống web. Giáo viên chọn chức năng tạo đề thi mới, nhập các thông tin cơ bản như tên đề thi, môn học, lớp học, thời gian thi và số câu hỏi. Tiếp theo, giáo viên chọn câu hỏi từ ngân hàng câu hỏi hoặc tạo câu hỏi mới. Hệ thống hỗ trợ cấu hình số lượng mã đề và tùy chọn xáo trộn câu hỏi, đáp án cho từng mã đề. Sau khi hoàn tất, giáo viên xuất bản đề thi để học sinh có thể tiếp cận.

Quy trình quét phiếu và chấm thi bắt đầu khi học sinh đăng nhập vào ứng dụng di động. Học sinh chọn bài thi muốn làm từ danh sách các bài thi đã được giao. Hệ thống hiển thị thông tin bài thi và yêu cầu học sinh chụp ảnh phiếu trả lời. Học sinh đặt phiếu trả lời trong khung hình và nhấn nút chụp. Hệ thống xử lý ảnh, nhận diện mã đề và các đáp án đã tô. Kết quả chấm được hiển thị ngay cho học sinh và đồng thời được lưu lên server để giáo viên theo dõi.



Hình 2.2: Biểu đồ Sequence cho quy trình quét phiếu OMR

2.3 Đặc tả chức năng

Phần này trình bày chi tiết các Use Case quan trọng nhất của hệ thống, bao gồm mô tả các luồng sự kiện chính, luồng thay thế và các điều kiện biên.

2.3.1 Đặc tả Use Case Quản lý đề thi

Bảng 2.2: Đặc tả Use Case UC01: Quản lý đề thi

Tên Use Case	Quản lý đề thi
Mã Use Case	UC01
Tác nhân chính	Giáo viên
Tác nhân phụ	Hệ thống
Mô tả	Cho phép giáo viên tạo mới, chỉnh sửa, xóa và xuất bản các đề thi trắc nghiệm. Hệ thống hỗ trợ sinh nhiều mã đề tự động và xáo trộn câu hỏi, đáp án.
Tiền điều kiện	Giáo viên đã đăng nhập thành công vào hệ thống web. Giáo viên có quyền tạo đề thi cho các lớp được phân công.
Hậu điều kiện thành công	Đề thi được tạo và lưu vào cơ sở dữ liệu với trạng thái nháp. Nếu giáo viên xuất bản, đề thi chuyển sang trạng thái công khai và học sinh có thể nhìn thấy.
Hậu điều kiện thất bại	Đề thi không được tạo, thông báo lỗi được hiển thị. Các thay đổi không được lưu.

Luồng sự kiện chính:

1. Giáo viên chọn chức năng “Tạo đề thi mới” từ menu chính.
2. Hệ thống hiển thị form nhập thông tin đề thi, bao gồm: tên đề thi, môn học (chọn từ danh sách), lớp học (chọn từ danh sách), thời gian thi (phút), số câu hỏi, ngày thi, và các tùy chọn khác.
3. Giáo viên nhập thông tin và nhấn “Tiếp tục”.
4. Hệ thống hiển thị giao diện chọn câu hỏi từ ngân hàng câu hỏi, với các bộ lọc theo môn học, chủ đề, độ khó.
5. Giáo viên chọn các câu hỏi từ ngân hàng hoặc tạo câu hỏi mới.
6. Giáo viên cấu hình số lượng mã đề và tùy chọn xáo trộn.
7. Giáo viên nhấn “Lưu nháp” hoặc “Xuất bản”.
8. Hệ thống tạo các mã đề theo cấu hình, lưu vào cơ sở dữ liệu và hiển thị thông báo thành công.

Luồng sự kiện phụ:

- 5a Nếu ngân hàng câu hỏi trống, hệ thống hiển thị thông báo và cho phép giáo viên tạo câu hỏi mới trước khi tiếp tục.
- 7a Nếu giáo viên chưa chọn đủ số câu hỏi, hệ thống hiển thị cảnh báo và yêu cầu bổ sung.
- 7b Nếu kết nối mạng bị gián đoạn, hệ thống lưu tạm và thông báo cho giáo viên tiếp tục sau.

2.3.2 Đặc tả Use Case Quét phiếu trả lời

Bảng 2.3: Đặc tả Use Case UC02: Quét phiếu trả lời

Tên Use Case	Quét phiếu trả lời
Mã Use Case	UC02
Tác nhân chính	Học sinh, Giáo viên
Tác nhân phụ	Hệ thống
Mô tả	Cho phép người dùng sử dụng camera của thiết bị di động để chụp ảnh phiếu trả lời trắc nghiệm. Hệ thống tự động xử lý ảnh, nhận diện mã đề, SBD và các đáp án đã tô, sau đó chấm điểm và hiển thị kết quả.
Tiền điều kiện	Người dùng đã đăng nhập vào ứng dụng mobile. Thiết bị có camera và được cấp quyền truy cập camera. Học sinh có quyền truy cập bài thi đã được giao.
Hậu điều kiện thành công	Kết quả chấm thi được hiển thị cho người dùng. Kết quả được lưu vào cơ sở dữ liệu với trạng thái đã chấm. Hình ảnh phiếu đã quét được lưu kèm kết quả.
Hậu điều kiện thất bại	Thông báo lỗi được hiển thị. Người dùng được yêu cầu thử lại hoặc điều chỉnh cách chụp.

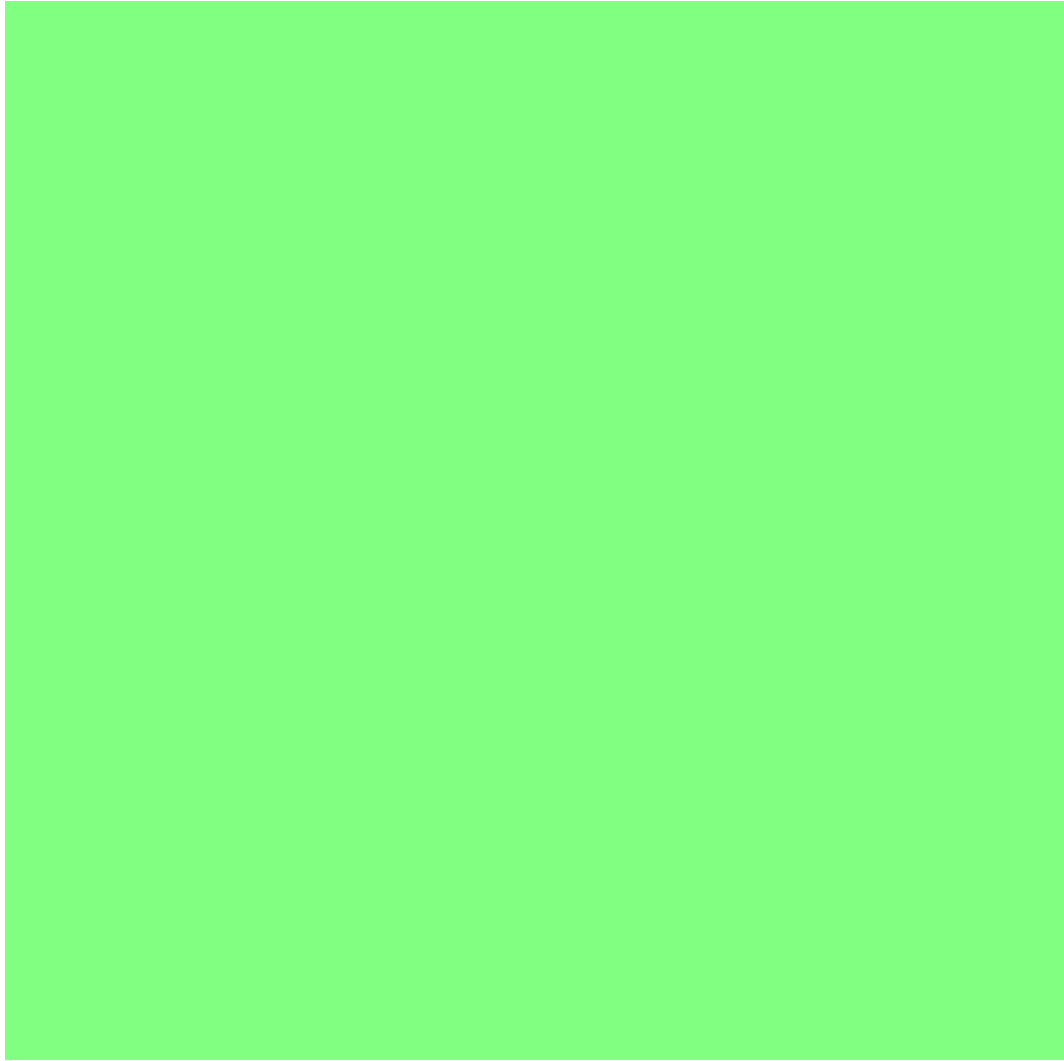
Luồng sự kiện chính:

1. Người dùng chọn bài thi muốn làm từ danh sách các bài thi.
2. Hệ thống hiển thị thông tin bài thi (tên, số câu, thời gian làm bài) và yêu cầu xác nhận.
3. Người dùng nhấn “Bắt đầu quét”.
4. Hệ thống mở camera và hiển thị khung hình xem trước với hướng dẫn đặt phiếu.
5. Người dùng đặt phiếu trả lời trong khung hình và nhấn nút chụp.
6. Hệ thống chụp ảnh và thực hiện tiền xử lý: chuyển sang grayscale, cân bằng histogram, khử nhiễu.
7. Hệ thống phát hiện các góc của phiếu bằng thuật toán Canny edge detection và tìm contours.

8. Hệ thống thực hiện perspective transform để căn chỉnh phiếu về đúng hướng.
9. Hệ thống nhận diện vùng chứa mã đề và SBD, trích xuất thông tin này.
10. Hệ thống nhận diện vùng chứa các đáp án, quét từng bubble và tính mật độ pixel.
11. Hệ thống đối chiếu đáp án với đáp án đúng của mã đề tương ứng và tính điểm.
12. Hệ thống hiển thị kết quả với điểm tổng, số câu đúng/sai, và danh sách chi tiết từng câu.
13. Hệ thống lưu kết quả và hình ảnh phiếu đã quét vào cơ sở dữ liệu.

Luồng sự kiện phụ:

- 6a Nếu ảnh chụp quá mờ hoặc thiếu sáng, hệ thống hiển thị cảnh báo và yêu cầu chụp lại với điều kiện ánh sáng tốt hơn.
- 8a Nếu hệ thống không tìm thấy đủ 4 góc của phiếu, hiển thị hướng dẫn căn chỉnh và yêu cầu chụp lại.
- 10a Nếu phát hiện tô đúp (2 bubble trở lên được tô trong cùng một câu), hệ thống đánh dấu cảnh báo và không tính điểm cho câu đó.
- 10b Nếu phát hiện bubble tô mờ (mật độ dưới ngưỡng), hệ thống hiển thị cảnh báo về độ tin cậy của kết quả.
- 13a Nếu kết nối mạng không khả dụng, hệ thống lưu kết quả vào bộ nhớ cục bộ và thực hiện đồng bộ khi có mạng.



Hình 2.3: Biểu đồ Use Case phân rã cho chức năng quét phiếu OMR

2.3.3 Đặc tả Use Case Xem kết quả thi

Bảng 2.4: Đặc tả Use Case UC03: Xem kết quả thi

Tên Use Case	Xem kết quả thi
Mã Use Case	UC03
Tác nhân chính	Học sinh, Giáo viên
Mô tả	Cho phép người dùng xem chi tiết kết quả bài thi, bao gồm điểm số, số câu đúng sai, thời gian nộp bài, và hình ảnh phiếu đã quét. Giáo viên có thể xem tổng hợp kết quả của tất cả học sinh trong lớp.
Tiền điều kiện	Người dùng đã đăng nhập. Đã có ít nhất một bài thi đã được chấm.
Hậu điều kiện thành công	Thông tin kết quả thi được hiển thị đầy đủ và chính xác.

Luồng sự kiện chính:

1. Người dùng chọn mục “Kết quả thi” hoặc “Lịch sử thi” từ menu.
2. Hệ thống hiển thị danh sách các bài thi đã có kết quả, sắp xếp theo ngày thi gần nhất.
3. Người dùng chọn một bài thi cụ thể.
4. Hệ thống hiển thị thông tin tổng quan: điểm số, xếp hạng (nếu có), thời gian nộp bài.
5. Người dùng có thể xem chi tiết từng câu hỏi với đáp án đã chọn và đáp án đúng.
6. Người dùng có thể xem lại hình ảnh phiếu đã quét.
7. Giáo viên có thể xuất danh sách kết quả ra file Excel/PDF.

2.4 Yêu cầu phi chức năng

Ngoài các yêu cầu chức năng đã được phân tích, hệ thống cần đáp ứng một số yêu cầu phi chức năng quan trọng để đảm bảo chất lượng và trải nghiệm người dùng.

2.4.1 Yêu cầu về hiệu năng

Về mặt hiệu năng xử lý, hệ thống cần đảm bảo thời gian xử lý trung bình cho mỗi phiếu trả lời không quá 2 giây, bao gồm toàn bộ quá trình từ chụp ảnh đến hiển thị kết quả. Đối với việc xử lý batch (nhiều phiếu cùng lúc), hệ thống cần có khả năng xử lý 100 phiếu trong thời gian không quá 5 phút. Độ chính xác nhận diện OMR phải đạt tối thiểu 95% trên các điều kiện ảnh thông thường, và tỷ lệ phát hiện tô đúp phải đạt tối thiểu 98%. Thời gian phản hồi của API khi truy vấn dữ liệu không được vượt quá 500 mili giây trong điều kiện bình thường.

Về khả năng mở rộng, hệ thống thiết kế để hỗ trợ tối thiểu 100 người dùng đồng thời mà không giảm hiệu suất đáng kể. Kiến trúc backend được thiết kế theo hướng microservices để dễ dàng mở rộng theo chiều ngang khi cần thiết.

2.4.2 Yêu cầu về bảo mật

Về xác thực và ủy quyền, tất cả các API của hệ thống đều yêu cầu xác thực bằng JSON Web Token (JWT). Mật khẩu người dùng được lưu trữ dưới dạng hash sử dụng thuật toán bcrypt với salt đủ độ dài. Hệ thống phân quyền theo mô hình RBAC (Role-Based Access Control), đảm bảo mỗi người dùng chỉ có thể truy cập các chức năng và dữ liệu được phép.

Về bảo mật dữ liệu, toàn bộ dữ liệu truyền tải giữa client và server được mã hóa bằng giao thức HTTPS. Các thông tin nhạy cảm như điểm thi, thông tin cá nhân được bảo vệ theo các tiêu chuẩn về quyền riêng tư trong giáo dục. Hệ thống ghi log các hoạt động quan trọng để phục vụ mục đích kiểm toán và phát hiện xâm nhập.

2.4.3 Yêu cầu về tính khả dụng

Về khả năng sẵn sàng, hệ thống cần đạt uptime tối thiểu 99,5% trong giờ làm việc, với kế hoạch bảo trì được thông báo trước tối thiểu 24 giờ. Đối với ứng dụng di động, hệ thống cần hoạt động được offline để đảm bảo tính sẵn sàng trong mọi điều kiện, với cơ chế đồng bộ dữ liệu khi có kết nối mạng.

Về tính tương thích, ứng dụng mobile cần hỗ trợ tối thiểu iOS 12.0 và Android 8.0 (API 26) trở lên. Ứng dụng web cần tương thích với các trình duyệt phổ biến bao gồm Chrome, Firefox, Safari và Edge ở các phiên bản mới nhất.

Về khả năng tiếp cận, giao diện người dùng được thiết kế theo nguyên tắc dễ sử dụng, với các hướng dẫn rõ ràng và phản hồi tức thì cho mọi thao tác. Hệ thống hỗ trợ tiếng Việt đầy đủ, bao gồm cả bảng mã Unicode và các ký tự đặc biệt của tiếng Việt.

Bảng 2.5: Tổng hợp yêu cầu phi chức năng của hệ thống

Loại	Tiêu chí	Yêu cầu cụ thể
Hiệu năng	Thời gian xử lý mỗi phiếu	≤ 2 giây
Hiệu năng	Thời gian xử lý batch 100 phiếu	≤ 5 phút
Hiệu năng	Độ chính xác OMR	$\geq 95\%$
Hiệu năng	Thời gian phản hồi API	$\leq 500\text{ms}$
Hiệu năng	Số user đồng thời	≥ 100 user
Bảo mật	Xác thực	JWT với token hết hạn
Bảo mật	Mã hóa mật khẩu	bcrypt hash
Bảo mật	Truyền tải dữ liệu	HTTPS
Bảo mật	Phân quyền	RBAC
Khả dụng	Uptime	$\geq 99,5\%$
Khả dụng	Offline mode	Đầy đủ trên mobile
Khả dụng	Tương thích mobile	iOS 12+, Android 8+
Khả dụng	Tương thích web	Chrome, Firefox, Safari, Edge

2.4.4 Yêu cầu về tính dễ bảo trì

Mã nguồn của hệ thống được tổ chức theo các chuẩn clean code và có documentation đầy đủ cho các module quan trọng. Hệ thống CI/CD được thiết lập để tự động chạy unit test và integration test mỗi khi có commit mới. Các log được ghi chi tiết ở các mức DEBUG, INFO, WARNING, ERROR để phục vụ việc theo dõi và xử lý sự cố.

Kết chương

Chương 2 đã trình bày kết quả khảo sát hiện trạng về quy trình chấm thi trắc nghiệm tại các trường học Việt Nam, xác định rõ các hạn chế như tốn thời gian, dễ sai sót và chi phí cao. Qua khảo sát các giải pháp hiện có trên thị trường, đề tài xác định được những ưu điểm cần kế thừa và những hạn chế cần khắc phục. Chương đã phân tích và đặc tả chi tiết các yêu cầu chức năng thông qua các biểu đồ Use Case và đặc tả 3 Use Case quan trọng nhất. Các yêu cầu phi chức năng về hiệu năng, bảo mật, tính khả dụng và dễ bảo trì cũng đã được xác định rõ ràng. Chương tiếp theo sẽ giới thiệu các

nền tảng lý thuyết và công nghệ được sử dụng để triển khai hệ thống theo các yêu cầu đã được xác định trong chương này.

Chương 3

CHƯƠNG 3: NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Tổng quan chương

Chương 2 đã phân tích chi tiết các yêu cầu chức năng và phi chức năng của hệ thống. Chương này giới thiệu các nền tảng lý thuyết và công nghệ được sử dụng để triển khai hệ thống SMART GRADING. Cụ thể, chương trình bày về công nghệ nhận diện đánh dấu quang học OMR và các kỹ thuật xử lý ảnh liên quan; giới thiệu framework Flutter cho phát triển ứng dụng di động đa nền tảng; trình bày React và TypeScript cho xây dựng ứng dụng web; và cuối cùng là Node.js với MongoDB cho backend API.

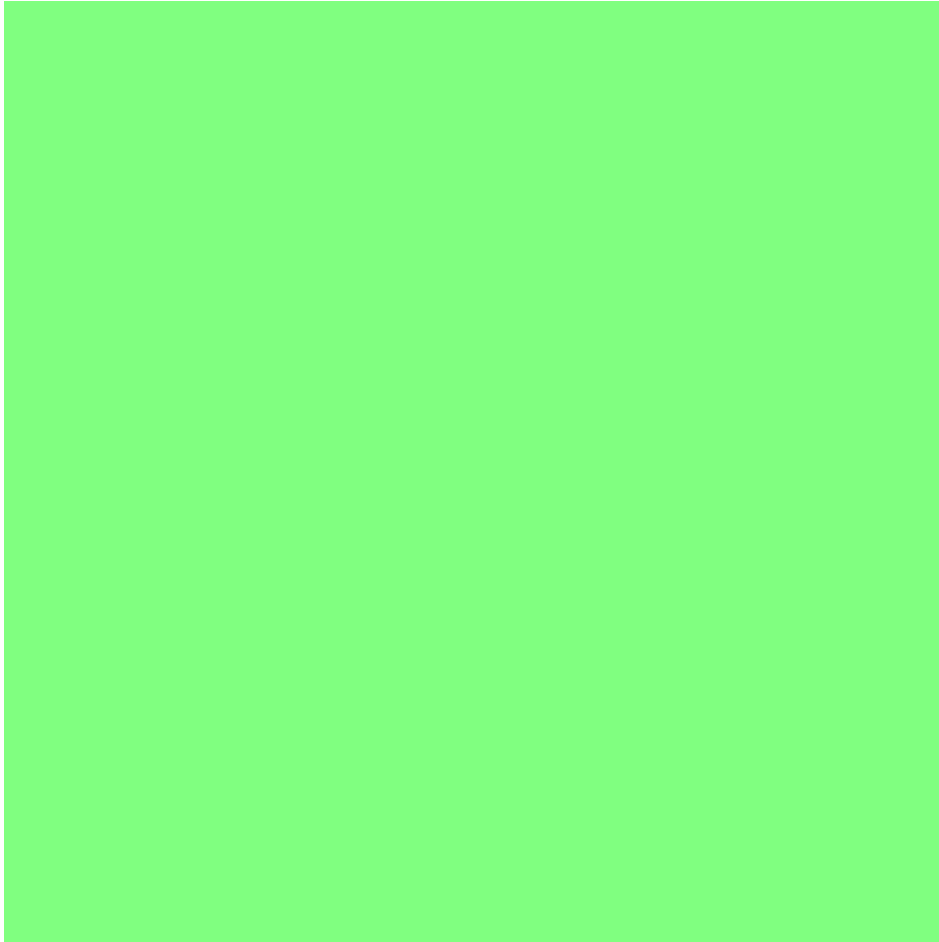
3.1 Công nghệ nhận diện đánh dấu quang học OMR

3.1.1 Giới thiệu về OMR

OMR (Optical Mark Recognition - Nhận diện đánh dấu quang học) là công nghệ cho phép nhận diện các vùng đã được đánh dấu trên giấy bằng bút chì, bút mực hoặc các công cụ đánh dấu khác. Khác với OCR (Optical Character Recognition) nhận diện ký tự, OMR tập trung vào việc phát hiện sự hiện diện hay vắng mặt của các vùng đánh dấu, dựa trên sự khác biệt về mật độ pixel giữa vùng được tô đậm và vùng trắng.

Nguyên lý hoạt động cơ bản của OMR dựa trên việc đo lường lượng ánh sáng phản xạ từ bề mặt giấy. Một vùng được tô đậm sẽ hấp thụ nhiều ánh sáng hơn và phản xạ ít hơn so với vùng trắng. Thiết bị đọc OMR sử dụng cảm biến quang để đo lường cường độ ánh sáng phản xạ này và chuyển đổi thành tín hiệu điện. Trong xử lý ảnh số, nguyên lý này được mô phỏng bằng cách phân tích mật độ pixel trong vùng quan tâm: các pixel đen (giá trị thấp) biểu thị vùng được tô, trong khi các pixel trắng (giá trị cao) biểu thị vùng trống.

OMR được ứng dụng rộng rãi trong nhiều lĩnh vực, đặc biệt là trong giáo dục cho việc chấm thi trắc nghiệm, trong khảo sát và thu thập dữ liệu, trong bỏ phiếu điện tử, và trong quản lý tài liệu. Ưu điểm của công nghệ OMR bao gồm tốc độ xử lý cao, độ chính xác ổn định, chi phí vật tư thấp (chỉ cần giấy in thông thường), và khả năng xử lý hàng loạt.



Hình 3.1: Luồng xử lý OMR từ thu ảnh đến trích xuất kết quả

3.1.2 Quy trình xử lý ảnh OMR

Quy trình xử lý ảnh OMR trong hệ thống SMART GRADING bao gồm các bước chính sau đây, mỗi bước đóng vai trò quan trọng trong việc đảm bảo độ chính xác của kết quả nhận diện.

Bước 1: Tiền xử lý ảnh (Preprocessing) là bước đầu tiên và rất quan trọng, nhằm chuẩn bị ảnh đầu vào ở định dạng tối ưu cho các bước xử lý tiếp theo. Đầu tiên, ảnh màu được chuyển đổi sang ảnh xám (grayscale) để giảm chiều phức tạp và tập trung vào thông tin cường độ sáng. Công thức chuyển đổi phổ biến là: $Y = 0.299 \times R + 0.587 \times G + 0.114 \times B$, trong đó các hệ số phản ánh độ nhạy của mắt người đối với các màu khác nhau. Tiếp theo, kỹ thuật cân bằng histogram được áp dụng để tăng cường độ tương phản của ảnh, đặc biệt hữu ích khi ảnh chụp có điều kiện ánh sáng không đồng đều. Cuối cùng, bộ lọc Gaussian blur với kernel 5×5 được sử dụng để giảm nhiễu trong ảnh mà không làm mất quá nhiều chi tiết cạnh.

Bước 2: Phát hiện góc tài liệu (Corner Detection) nhằm xác định vị trí của phiếu trả lời trong ảnh và các điểm góc của nó. Thuật toán Canny edge detection được áp dụng với các ngưỡng $\text{threshold1} = 50$ và $\text{threshold2} = 150$ để tìm các cạnh trong ảnh. Sau đó, thuật toán tìm contours được sử dụng để xác định các vùng khép kín trong ảnh. Từ danh sách contours, hệ thống lọc ra contours có diện tích lớn nhất và có dạng gần với hình chữ nhật. Điểm góc được xác định bằng thuật toán Approximate PolyDP để xấp xỉ đa giác với độ chính xác phù hợp.

Bước 3: Căn chỉnh ảnh (Perspective Transform) sử dụng ma trận transform được tính toán từ 4 điểm góc để biến đổi ảnh về dạng nhìn từ trên xuống (top-down view). Bước này đảm bảo rằng phiếu trả lời được hiển thị ở góc nhìn thẳng, loại bỏ các biến dạng do góc chụp nghiêng hoặc cong vênh của giấy. Ảnh sau transform có kích thước cố định (ví dụ: 2480 x 3508 pixel tương ứng với A4 ở 300 DPI), giúp việc xác định vị trí các vùng quan tâm trở nên nhất quán và chính xác hơn.

Bước 4: Xác định vùng câu trả lời (ROI Detection) dựa trên cấu hình template của phiếu OMR. Template định nghĩa vị trí chính xác của các vùng chứa thông tin (mã đề, SBD) và các vùng chứa câu trả lời. Hệ thống crop các vùng này từ ảnh đã căn chỉnh để xử lý riêng, giảm noise và tăng độ chính xác.

Bước 5: Phân tích từng bubble là bước cốt lõi trong quy trình OMR. Với mỗi vùng bubble, hệ thống tính tổng giá trị pixel và mật độ pixel theo công thức: $D_b = \frac{1}{N \times 255} \sum_{i=1}^N p_i$, trong đó D_b là mật độ của bubble thứ b , N là tổng số pixel trong vùng, và p_i là giá trị pixel thứ i (0-255). Mật độ này nằm trong khoảng từ 0 (hoàn toàn đen) đến 1 (hoàn toàn trắng). Một bubble được coi là đã tô nếu mật độ thấp hơn ngưỡng threshold T (thường là 0,5). Trong mỗi nhóm bubble (câu hỏi), bubble có mật độ thấp nhất (tô đậm nhất) được chọn làm đáp án.

Bước 6: Trích xuất và kiểm tra đáp án bao gồm việc map vị trí bubble sang đáp án (A, B, C, D hoặc các ký hiệu khác), kiểm tra các trường hợp đặc biệt như không có bubble nào được tô (bỏ trống) hoặc nhiều bubble có mật độ tương đương (tô kép), và tạo ma trận đáp án cuối cùng.

3.1.3 Các kỹ thuật xử lý ảnh nâng cao

Để đạt được độ chính xác cao trong điều kiện ảnh không lý tưởng, hệ thống SMART GRADING áp dụng một số kỹ thuật xử lý ảnh nâng cao.

Adaptive Threshold (Ngưỡng thích ứng) là kỹ thuật cho phép điều chỉnh ngưỡng nhị phân dựa trên đặc điểm cục bộ của ảnh. Thay vì sử dụng một ngưỡng cố định cho toàn bộ ảnh, adaptive threshold tính ngưỡng riêng cho từng vùng nhỏ dựa trên giá trị trung bình hoặc trung vị của các pixel xung quanh. Điều này đặc biệt hữu ích khi ảnh có điều kiện ánh sáng không đồng đều, ví dụ như một phần của phiếu bị đổ bóng. Công thức tính ngưỡng cho vùng nhỏ là: $T(x, y) = \text{mean}(I_{\text{region}}) - C$, trong đó C là hằng số điều chỉnh.

Contour Analysis được sử dụng để xác định hình dạng và vị trí của các đối tượng trong ảnh. Hệ thống sử dụng thuật toán Suzuki-Abe để tìm contours, sau đó phân tích các thuộc tính như diện tích, chu vi, tỷ lệ aspect ratio để lọc ra các contours không phải là bubble. Các contours có diện tích và hình dạng phù hợp với kích thước bubble được xác định trong template sẽ được giữ lại để phân tích tiếp.

Perspective Transform sử dụng ma trận biến đổi 3x3 (homography matrix) để ánh xạ các điểm ảnh từ góc nhìn nghiêng về góc nhìn thẳng. Ma trận này được tính toán dựa trên 4 điểm góc đã xác định và 4 điểm đích tương ứng. OpenCV cung cấp hàm `getPerspectiveTransform()` để tính ma trận và `warpPerspective()` để áp dụng biến đổi.

Bảng 3.1: Các kỹ thuật xử lý ảnh OMR và ứng dụng

Kỹ thuật	Mục đích	Tham số
Grayscale conversion	Giảm chiều phức tạp, tập trung cường độ sáng	-
Histogram equalization	Tăng tương phản, cân bằng sáng	-
Gaussian blur	Khử nhiễu	kernel=5x5, sigma=1.5
Canny edge detection	Tìm cạnh trong ảnh	th1=50, th2=150
Contour detection	Xác định vùng hình chữ nhật	minArea, maxArea
Approximate PolyDP	Xấp xỉ đa giác, tìm góc	epsilon=0.02
Perspective transform	Căn chỉnh góc nhìn	4 điểm góc
Adaptive threshold	Xử lý ảnh không đều sáng	blockSize=11, C=2
Pixel density analysis	Đo mức độ tô của bubble	threshold=0.5

3.2 Framework Flutter cho ứng dụng di động

3.2.1 Giới thiệu về Flutter

Flutter là framework phát triển ứng dụng di động đa nền tảng được Google phát triển và ra mắt lần đầu vào năm 2017. Flutter cho phép xây dựng ứng dụng native cho iOS và Android từ một codebase duy nhất, sử dụng ngôn ngữ lập trình Dart. Điểm nổi bật của Flutter so với các framework cross-platform khác là việc sử dụng engine đồ họa Skia để vẽ trực tiếp các widget, thay vì sử dụng các thành phần native của hệ điều hành. Điều này đảm bảo hiệu năng gần như ứng dụng native thực sự.

Flutter có nhiều ưu điểm nổi bật khiến nó trở thành lựa chọn phù hợp cho dự án SMART GRADING. Tính năng Hot Reload cho phép nhà phát triển xem thay đổi ngay lập tức trong quá trình phát triển mà không cần build lại toàn bộ ứng dụng, giúp tăng đáng kể năng suất làm việc. Hệ thống widget của Flutter rất phong phú và có thể tùy chỉnh cao, bao gồm Material Design và Cupertino widgets, cho phép xây dựng giao diện đẹp mắt và nhất quán trên cả hai nền tảng. Ngoài ra, Flutter có cộng đồng phát triển đông đảo và đang tăng trưởng nhanh, với nhiều package hỗ trợ trên pub.dev.

Trong dự án SMART GRADING, Flutter được sử dụng để xây dựng ứng dụng cho phép người dùng quét phiếu trả lời trắc nghiệm bằng camera, xem kết quả chấm thi, và theo dõi lịch sử các bài thi. Ứng dụng cần hoạt động mượt mà trên nhiều loại thiết bị với cấu hình khác nhau, từ điện thoại tầm trung đến máy tính bảng.

3.2.2 Kiến trúc ứng dụng Flutter

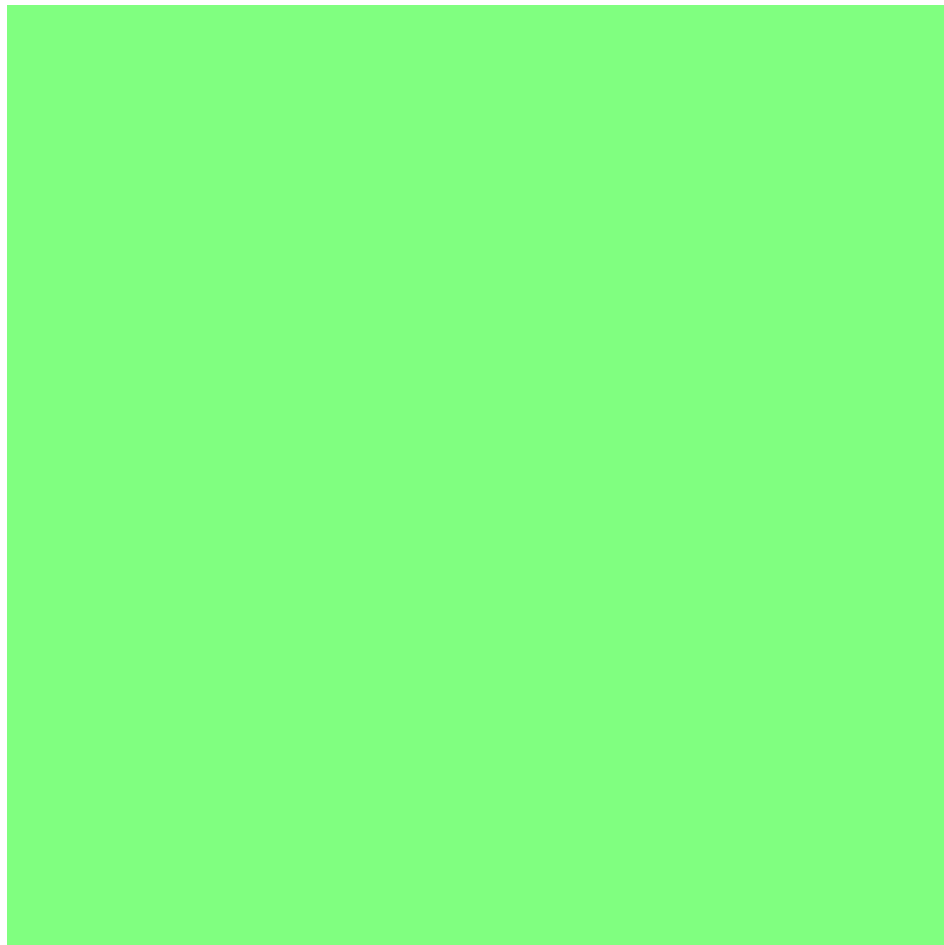
Ứng dụng Flutter trong dự án SMART GRADING được thiết kế theo kiến trúc Clean Architecture, phân tách rõ ràng các tầng để đảm bảo tính module hóa, dễ bảo trì và khả năng kiểm thử.

Tầng Presentation (Trình bày) bao gồm các widget và màn hình (page) là giao diện người dùng trực tiếp. Các widget được xây dựng từ các thành phần cơ bản của Flutter như Container, Text, Button, và được tổ chức theo nguyên tắc composition (thành

phần). Mỗi màn hình sử dụng các widget này để xây dựng giao diện hoàn chỉnh và kết nối với BLoC để quản lý trạng thái.

Tầng Business Logic (Logic nghiệp vụ) được triển khai bằng pattern BLoC (Business Logic Component). BLoC nhận các sự kiện (event) từ tầng presentation, xử lý logic nghiệp vụ, và phát ra các trạng thái (state) mới. Mô hình unidirectional data flow đảm bảo rằng dữ liệu luôn chảy theo một hướng duy nhất, giúp dễ dàng theo dõi và debug.

Tầng Data (Dữ liệu) bao gồm các repository, data source, và model. Repository định nghĩa các interface cho việc truy cập dữ liệu, che giấu chi tiết triển khai bên dưới. Data source có thể là remote (gọi API) hoặc local (lưu trữ trong SharedPreferences hoặc SQLite). Model là các class định nghĩa cấu trúc dữ liệu.



Hình 3.2: Kiến trúc Clean Architecture trong ứng dụng Flutter

3.2.3 State Management với flutter_bloc

Trong dự án SMART GRADING, package flutter_bloc được sử dụng để quản lý trạng thái ứng dụng. flutter_bloc là implementation của BLoC pattern cho Flutter, được phát triển dựa trên các nguyên tắc của reactive programming.

Một BLoC bao gồm ba thành phần chính: Event, State, và Bloc class. Event đại diện cho các sự kiện đầu vào từ người dùng hoặc hệ thống, như LoginRequested, ScanButtonPressed, hoặc SubmissionLoaded. State đại diện cho trạng thái hiện tại của một phần giao diện, như Loading, Success, hoặc Error. Bloc class là cầu nối, nhận event, xử lý logic, và phát ra state mới.

Ví dụ về một BLoC đơn giản trong hệ thống:

```
// Event: ScanButtonPressed
class ScanButtonPressed extends OMRScannerEvent {
  final String examId;
  ScanButtonPressed({required this.examId});
}

// State: Scanning, ScanSuccess, ScanError
class ScanSuccess extends OMRScannerState {
  final SubmissionResult result;
  ScanSuccess({required this.result});
}

// Bloc
class OMRScannerBloc
  extends Bloc<OMRScannerEvent, OMRScannerState> {

  OMRScannerBloc({
    required this.omrService,
    required this.examRepository,
  }) : super(OMRScannerInitial()) {
    on<ScanButtonPressed>(_onScanButtonPressed);
  }

  Future<void> _onScanButtonPressed(
    ScanButtonPressed event,
    Emitter<OMRScannerState> emit,
  ) async {
    emit(Scanning());
    try {
      final result = await omrService.scanAndGrade(event.examId);
      emit(ScanSuccess(result: result));
    } catch (e) {
      emit(ScanError(message: e.toString()));
    }
  }
}
```

3.3 React và TypeScript cho ứng dụng web

3.3.1 Giới thiệu về React

React là thư viện JavaScript để xây dựng giao diện người dùng, được phát triển bởi Facebook và ra mắt lần đầu vào năm 2013. React sử dụng mô hình component-based, cho phép xây dựng giao diện từ các thành phần độc lập và có thể tái sử dụng. Mỗi component quản lý trạng thái (state) riêng của mình và có thể lồng nhau để tạo thành giao diện phức tạp.

Điểm nổi bật của React là Virtual DOM - một biểu diễn ảo của DOM thực trong bộ nhớ. Khi state của component thay đổi, React so sánh Virtual DOM mới với Virtual DOM cũ (diffing), tính toán các thay đổi tối thiểu cần thiết, và chỉ cập nhật các phần thực sự thay đổi trong DOM thực. Điều này giúp React đạt được hiệu năng cao mà không cần re-render toàn bộ trang.

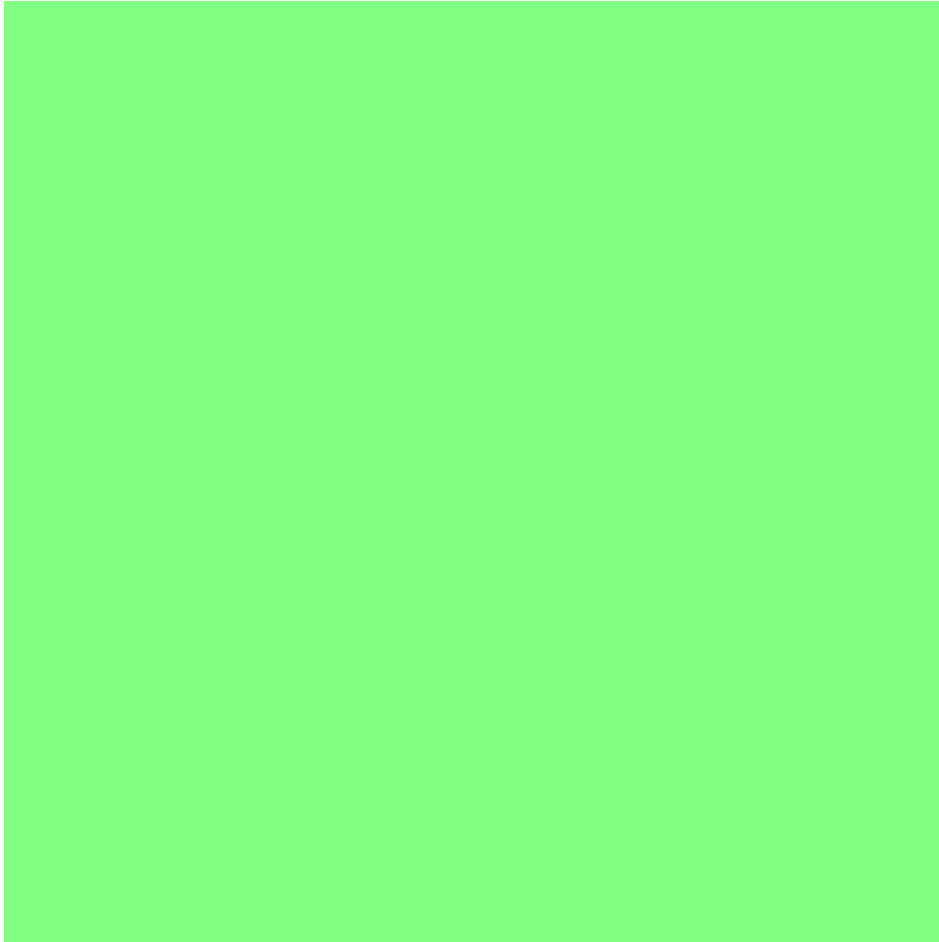
React được chọn cho dự án SMART GRADING vì nhiều lý do. React có hệ sinh thái phong phú với nhiều thư viện hỗ trợ cho state management (Redux, Zustand), routing (React Router), và data fetching (TanStack Query). React có cộng đồng rất lớn và tài liệu phong phú, giúp giải quyết vấn đề nhanh chóng. Ngoài ra, React được sử dụng rộng rãi trong ngành công nghiệp, đảm bảo tính ổn định và khả năng bảo trì lâu dài.

TypeScript là một superset của JavaScript được phát triển bởi Microsoft, bổ sung kiểm tra kiểu tĩnh (static type checking). TypeScript giúp phát hiện lỗi sớm trong quá trình phát triển, cải thiện khả năng đọc và bảo trì mã nguồn, và hỗ trợ các tính năng JavaScript hiện đại cùng với các tính năng đặc thù của riêng như interface, generic, enum.

3.3.2 Kiến trúc ứng dụng React

Ứng dụng web SMART GRADING được tổ chức theo cấu trúc thư mục feature-based, nhóm các file liên quan đến một tính năng cùng nhau thay vì nhóm theo loại file.

Thư mục src chứa toàn bộ mã nguồn, được phân chia thành các thư mục con theo chức năng. Thư mục components chứa các component có thể tái sử dụng ở nhiều nơi trong ứng dụng, như Button, Modal, LoadingSpinner. Thư mục pages chứa các component tương ứng với một trang cụ thể, như DashboardPage, ExamListPage, ExamDetailPage. Thư mục stores chứa các Zustand store cho quản lý trạng thái ứng dụng. Thư mục services chứa các service gọi API, như examService, questionService, submissionService. Thư mục hooks chứa các custom hook sử dụng chung, như useAuth, useExam. Thư mục types chứa các định nghĩa TypeScript interface và type.



Hình 3.3: Cấu trúc thư mục ứng dụng React

3.3.3 State Management với Zustand

Zustand là thư viện quản lý trạng thái cho React, được chọn cho dự án SMART GRADING nhờ tính đơn giản và hiệu năng cao. Khác với Redux yêu cầu boilerplate nhiều, Zustand cho phép tạo store chỉ với một vài dòng code.

Ví dụ về một Zustand store trong hệ thống:

```
// Store cho quản lý đề thi
interface ExamStore {
  exams: Exam[];
  currentExam: Exam | null;
  loading: boolean;

  fetchExams: () => Promise<void>;
  createExam: (data: CreateExamDTO) => Promise<void>;
  setCurrentExam: (exam: Exam) => void;
}

const useExamStore = create<ExamStore>((set, get) => ({
  exams: [],
  currentExam: null,
```

```

loading: false,

fetchExams: async () => {
  set({ loading: true });
  try {
    const exams = await examService.getAll();
    set({ exams, loading: false });
  } catch (error) {
    set({ loading: false });
    throw error;
  }
},

createExam: async (data) => {
  const newExam = await examService.create(data);
  set({ exams: [...get().exams, newExam] });
},

setCurrentExam: (exam) => set({ currentExam: exam }),
}));

```

3.4 Node.js và MongoDB cho Backend

3.4.1 Giới thiệu về Node.js

Node.js là runtime JavaScript được xây dựng trên engine V8 của Chrome, cho phép chạy mã JavaScript ở phía server. Node.js sử dụng mô hình I/O non-blocking (bất đồng bộ), đặc biệt hiệu quả cho các ứng dụng cần xử lý nhiều kết nối đồng thời như ứng dụng web real-time hoặc API server.

Ưu điểm của Node.js bao gồm khả năng sử dụng JavaScript trên toàn bộ stack phát triển (frontend và backend), giúp đơn giản hóa việc chuyển đổi ngữ cảnh và cho phép chia sẻ code giữa client và server. Mô hình event-driven và non-blocking I/O của Node.js đạt hiệu năng cao trong việc xử lý nhiều request đồng thời. Ngoài ra, Node.js có hệ sinh thái npm với hàng trăm nghìn package sẵn có, giúp tăng năng suất phát triển.

Express.js là framework web phổ biến nhất cho Node.js, cung cấp một lớp trừu tượng đơn giản và linh hoạt cho việc xây dựng API và web server. Express cung cấp các tính năng cốt lõi như routing, middleware, template engine, và error handling, trong khi vẫn đủ nhẹ và linh hoạt để tùy chỉnh theo nhu cầu.

Trong dự án SMART GRADING, Node.js với Express được sử dụng để xây dựng RESTful API, xử lý các yêu cầu từ ứng dụng Flutter và React, thực hiện logic nghiệp vụ, và tương tác với cơ sở dữ liệu MongoDB.

3.4.2 Giới thiệu về MongoDB

MongoDB là cơ sở dữ liệu NoSQL document-oriented, lưu trữ dữ liệu dưới dạng các document JSON (BSON). Khác với cơ sở dữ liệu quan hệ truyền thống sử dụng bảng

và hàng, MongoDB sử dụng collection và document, cho phép lưu trữ dữ liệu có cấu trúc linh hoạt và phù hợp với dữ liệu dạng JSON.

Mongoose là thư viện ODM (Object Data Mapping) cho MongoDB trong Node.js, cung cấp một lớp trừu tượng để làm việc với MongoDB một cách có cấu trúc và type-safe. Mongoose cho phép định nghĩa schema cho các document, thực hiện validation tự động, và hỗ trợ các tính năng như middleware, virtual property, và instance method.

```
// Ví dụ về Mongoose Schema cho Exam
const examSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    trim: true,
    maxlength: 200,
  },
  subject: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Subject',
    required: true,
  },
  classIds: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Class',
  }],
  questionIds: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Question',
  }],
  totalScore: {
    type: Number,
    required: true,
    min: 0,
  },
  duration: {
    type: Number, // minutes
    required: true,
  },
  status: {
    type: String,
    enum: ['draft', 'published', 'in_progress', 'completed'],
    default: 'draft',
  },
  numberOfVersions: {
    type: Number,
    default: 1,
    min: 1,
    max: 20,
  },
  examDate: {
```

```

    type: Date,
  },
  createdBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
  },
}, {
  timestamps: true,
});

// Instance method
examSchema.methods.publish = function() {
  this.status = 'published';
  return this.save();
};

// Static method
examSchema.statics.findByClass = function(classId) {
  return this.find({ classIds: classId });
};

export const Exam = mongoose.model('Exam', examSchema);

```

3.4.3 Kiến trúc Backend

Backend SMART GRADING được thiết kế theo kiến trúc phân tầng (layered architecture), phân tách rõ ràng các tầng để đảm bảo tính module hóa và khả năng bảo trì.

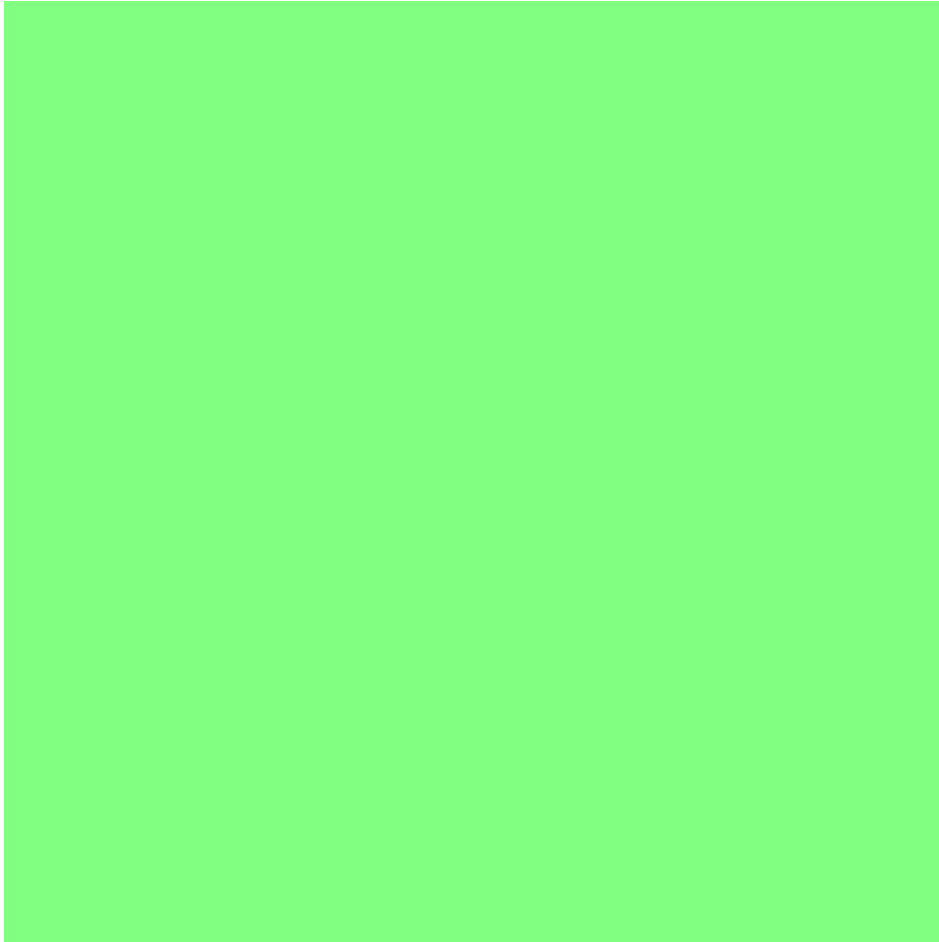
Tầng Routes định nghĩa các endpoint API và ánh xạ chúng đến các controller tương ứng. Routes cũng xử lý validation cơ bản của request parameters và body.

Tầng Controller nhận request từ tầng Routes, gọi các service tương ứng, và trả về response. Controller không chứa logic nghiệp vụ phức tạp, mà chỉ là cầu nối giữa HTTP layer và business logic.

Tầng Service chứa toàn bộ logic nghiệp vụ của hệ thống. Service tương tác với cơ sở dữ liệu thông qua các Model, thực hiện các phép tính, và đảm bảo các ràng buộc nghiệp vụ được áp dụng.

Tầng Model định nghĩa cấu trúc dữ liệu và các phương thức liên quan đến database. Mỗi model đại diện cho một collection trong MongoDB.

Middleware được sử dụng để xử lý các logic chung áp dụng cho nhiều route, như authentication, authorization, logging, và error handling.



Hình 3.4: Kiến trúc backend Node.js theo phân tầng

3.5 Lựa chọn và so sánh công nghệ

Trong quá trình nghiên cứu và phát triển hệ thống SMART GRADING, nhóm đã thực hiện khảo sát và so sánh nhiều công nghệ khác nhau để đưa ra lựa chọn phù hợp nhất cho từng thành phần.

Bảng 3.2: So sánh các lựa chọn công nghệ

Thành phần	Chọn	Alt 1	Alt 2
Mobile Framework	Flutter	React Native	Native (Swift/Kotlin)
Mobile Language	Dart	JavaScript	Swift/Kotlin
Web Framework	React	Vue.js	Angular
Web Language	TypeScript	JavaScript	TypeScript
Backend Runtime	Node.js	Python/Django	Go
Database	MongoDB	PostgreSQL	MySQL
Image Processing	OpenCV	PIL/Pillow	TensorFlow Lite
State Management	BLoC (Flutter)	Provider	Redux
API Protocol	REST	GraphQL	gRPC
Auth Method	JWT	OAuth 2.0	Session

Về Mobile Framework, Flutter được chọn thay vì React Native vì hiệu năng cao hơn nhờ compiled code thay vì JavaScript bridge, đồng thời có hệ thống widget phong phú và khả năng tùy chỉnh cao. Flutter cũng có Hot Reload nhanh hơn và được hỗ trợ tốt hơn bởi Google. Native development với Swift/Kotlin bị loại do yêu cầu phải duy trì hai codebase riêng biệt.

Về Web Framework, React được chọn thay vì Vue.js và Angular vì có hệ sinh thái lớn nhất, cộng đồng đông đảo nhất, và được sử dụng rộng rãi trong ngành công nghiệp. TypeScript được chọn thay vì JavaScript thuần vì lợi ích về kiểm tra kiểu tĩnh và khả năng bảo trì tốt hơn.

Về Backend, Node.js được chọn vì cho phép sử dụng JavaScript trên toàn bộ stack, đồng thời có hiệu năng tốt cho các ứng dụng I/O-intensive như API server. Python/Django bị loại do hiệu năng thấp hơn cho request-response cycle ngắn. Go có hiệu năng cao nhưng cộng đồng và thư viện hỗ trợ ít hơn.

Về Database, MongoDB được chọn vì phù hợp với dữ liệu có cấu trúc linh hoạt như đề thi và câu hỏi, đồng thời Mongoose ODM cung cấp interface type-safe tốt. PostgreSQL với JSON column có thể là lựa chọn thay thế nhưng MongoDB native JSON storage hiệu quả hơn.

Về Image Processing, OpenCV được chọn vì thư viện mạnh mẽ nhất cho xử lý ảnh với nhiều thuật toán được tối ưu hóa. PIL/Pillow chỉ phù hợp cho các tác vụ đơn giản. TensorFlow Lite có thể hữu ích cho nhận diện phức tạp hơn trong tương lai nhưng không cần thiết cho OMR cơ bản.

Kết chương

Chương 3 đã trình bày chi tiết các nền tảng lý thuyết và công nghệ được sử dụng để triển khai hệ thống SMART GRADING. Về công nghệ nhận diện OMR, chương đã giải thích nguyên lý hoạt động và quy trình xử lý ảnh từ tiền xử lý, phát hiện góc, căn chỉnh, đến phân tích bubble. Về phát triển ứng dụng di động, Flutter với kiến trúc Clean Architecture và BLoC pattern được lựa chọn và giải thích chi tiết. Về phát triển ứng dụng web, React với TypeScript và Zustand được trình bày. Về backend, Node.js với Express và MongoDB được phân tích cùng với kiến trúc phân tầng. Cuối cùng, chương tổng hợp các lựa chọn công nghệ và so sánh với các phương án thay thế. Chương tiếp theo sẽ trình bày chi tiết về phân tích thiết kế và triển khai hệ thống.

Chương 4

CHƯƠNG 4: PHÂN TÍCH THIẾT KẾ VÀ TRIỂN KHAI

Tổng quan chương

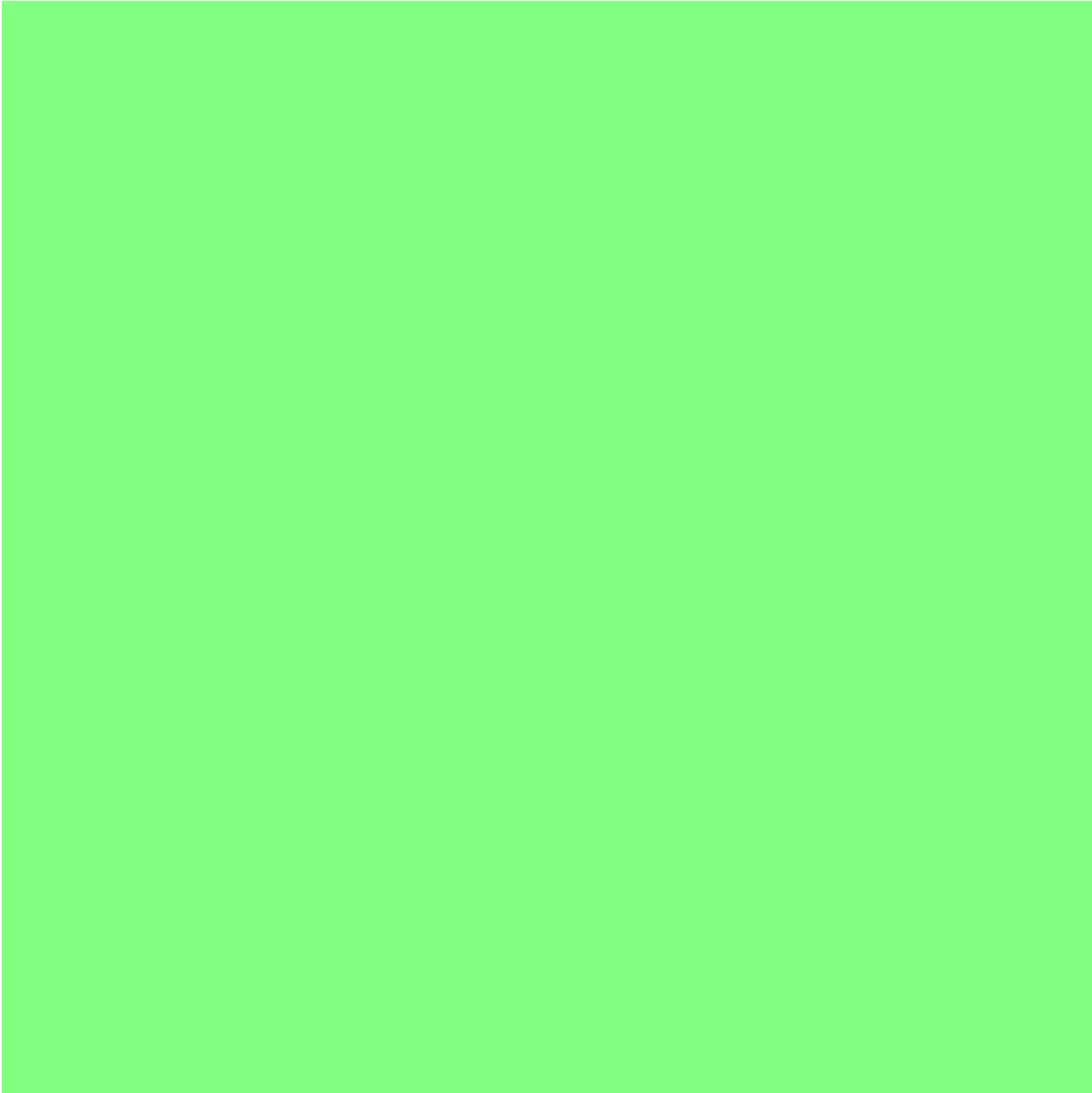
Chương 3 đã giới thiệu các nền tảng lý thuyết và công nghệ được sử dụng trong đề tài. Chương này trình bày chi tiết về phân tích và thiết kế hệ thống SMART GRADING, bao gồm kiến trúc tổng thể, thiết kế cơ sở dữ liệu, thiết kế các thành phần chính, và cuối cùng là trình bày kết quả triển khai cùng với các công cụ và thống kê liên quan.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống SMART GRADING được thiết kế theo kiến trúc Client-Server phân tầng (Layered Client-Server Architecture), kết hợp với các mẫu thiết kế Microservices cho một số thành phần mở rộng. Kiến trúc này được lựa chọn dựa trên các tiêu chí về khả năng mở rộng, dễ bảo trì, tính độc lập giữa các thành phần, và phù hợp với đặc điểm của các nền tảng được sử dụng.

Kiến trúc tổng thể của hệ thống gồm ba tầng chính. Tầng Client (Presentation Layer) bao gồm các ứng dụng tương tác trực tiếp với người dùng, bao gồm ứng dụng di động Flutter cho iOS và Android, ứng dụng web React cho trình duyệt trên máy tính, và trang quản trị cho admin. Tầng Business Logic (Application Layer) chứa toàn bộ logic nghiệp vụ của hệ thống, được triển khai dưới dạng RESTful API bằng Node.js và Express. Tầng Data (Data Layer) quản lý việc lưu trữ và truy xuất dữ liệu, sử dụng MongoDB cho dữ liệu chính và Cloudinary cho lưu trữ hình ảnh.



Hình 4.1: Kiến trúc tổng quan hệ thống SMART GRADING

Mô hình giao tiếp giữa các tầng sử dụng giao thức HTTP/HTTPS với định dạng JSON cho request và response. Các ứng dụng client giao tiếp với backend thông qua RESTful API, đảm bảo tính độc lập giữa client và server. Authentication được xử lý bằng JWT (JSON Web Token), cho phép stateless authentication và dễ dàng mở rộng.

4.1.2 Kiến trúc Mobile App (Flutter)

Ứng dụng di động Flutter được thiết kế theo kiến trúc Clean Architecture, phân tách rõ ràng ba tầng chính. Tầng Presentation chứa các widget và BLoC, quản lý giao diện người dùng và trạng thái ứng dụng. Tầng Domain chứa các entity và use case, định nghĩa các nghiệp vụ cốt lõi mà không phụ thuộc vào chi tiết triển khai. Tầng Data chứa các repository implementation, data source, và model, xử lý việc truy xuất dữ liệu từ API hoặc local storage.

Cấu trúc thư mục của ứng dụng Flutter được tổ chức theo nguyên tắc feature-first, nhóm các file liên quan đến một tính năng cùng nhau. Thư mục core chứa các cấu

hình chung như constants, theme, network client. Thư mục domain chứa các entity và repository interface. Thư mục data chứa model và repository implementation. Thư mục presentation chứa BLoC, pages, và widgets.

Đặc biệt, thư mục domain/omr chứa engine xử lý OMR được triển khai hoàn toàn bằng Dart, cho phép xử lý ảnh trực tiếp trên thiết bị mà không cần gọi đến backend. Điều này đảm bảo tính offline và giảm tải cho server.

```
lib/
+--- core/
|   +--- constants/          # App-wide constants
|   +--- network/            # API client, endpoints
|   +--- theme/              # Theme configuration
|   +--- utils/              # Helper functions
|
+--- domain/
|   +--- entities/           # Business entities
|   +--- repositories/       # Repository interfaces
|   +--- omr/                # OMR Engine
|   |   +--- models/         # OMR data models
|   |   +--- engine.dart     # Main engine
|   |   +--- preprocessor.dart # Image preprocessing
|   |   +--- detector.dart   # Corner/document detection
|   |   +--- scanner.dart    # Bubble reading
|   +--- usecases/           # Use cases
|
+--- data/
|   +--- models/             # Data models (DTO)
|   +--- repositories/       # Repository implementations
|   +--- datasources/        # Remote/local data sources
|
+--- presentation/
|   +--- blocs/              # BLoC state management
|   +--- pages/              # Screen widgets
|   +--- widgets/           # Reusable widgets
|
+--- main.dart
```

4.1.3 Kiến trúc Web App (React)

Ứng dụng web React được thiết kế theo cấu trúc SPA (Single Page Application), cho phép trải nghiệm người dùng liền mạch mà không cần reload trang. React Router được sử dụng để quản lý navigation, với protected routes cho các trang yêu cầu authentication.

State management được tổ chức bằng Zustand, với các store được phân chia theo domain như authStore, examStore, questionStore, submissionStore. TanStack Query được sử dụng cho việc quản lý server state, cung cấp caching tự động, background refetch, và optimistic updates.

Cấu trúc thư mục của ứng dụng web theo mô hình feature-based, nhóm các file liên quan đến một tính năng cùng nhau thay vì theo loại file. Điều này giúp dễ dàng mở rộng và bảo trì khi ứng dụng phát triển lớn.

```
src/
+-- components/           # Shared components
|   +-- Layout/
|   +-- Common/
|   +-- Admin/
|
+-- pages/                # Page components
|   +-- auth/
|   +-- dashboard/
|   +-- exams/
|   +-- questions/
|   +-- submissions/
|   +-- admin/
|
+-- presentation/
|   +-- store/            # Zustand stores
|   +-- hooks/            # Custom hooks
|   +-- routes/           # Route configuration
|
+-- services/             # API services
|   +-- auth.service.ts
|   +-- exam.service.ts
|   +-- ...
|
+-- types/                # TypeScript types
|
+-- App.tsx
```

4.1.4 Kiến trúc Backend (Node.js)

Backend được thiết kế theo kiến trúc phân tầng (Layered Architecture) với các thành phần rõ ràng. Routes tiếp nhận HTTP request và dispatch đến controller tương ứng. Controllers xử lý HTTP layer, validate input, và gọi service. Services chứa logic nghiệp vụ, tương tác với database và external services. Models định nghĩa schema và methods cho database documents.

Middleware được sử dụng cho các cross-cutting concerns như authentication, authorization, logging, validation, và error handling. Error handling middleware đảm bảo tất cả errors được catch và trả về response consistent.

```
server/src/
+-- config/
|   +-- index.js          # App configuration
|   +-- logger.js         # Logging setup
|
+-- controllers/
```

```

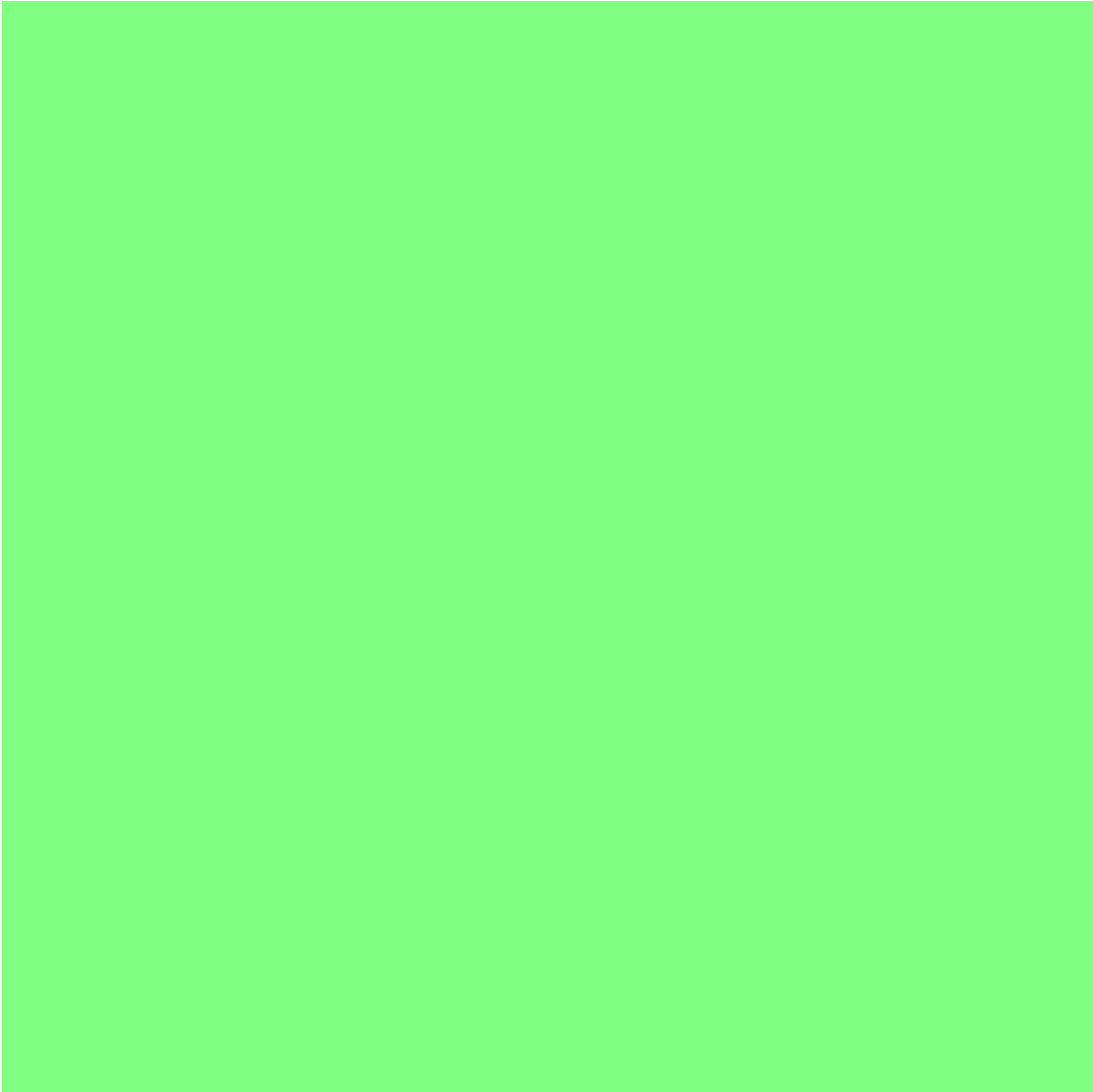
|   +-- auth.controller.js
|   +-- user.controller.js
|   +-- exam.controller.js
|   +-- question.controller.js
|   +-- submission.controller.js
|   +-- ...
|
+-- services/
|   +-- auth.service.js
|   +-- user.service.js
|   +-- exam.service.js
|   +-- omr.service.js
|   +-- report.service.js
|   +-- ...
|
+-- models/
|   +-- user.model.js
|   +-- exam.model.js
|   +-- question.model.js
|   +-- submission.model.js
|   +-- ...
|
+-- routes/
|   +-- v1/
|       +-- index.js
|       +-- auth.route.js
|       +-- exam.route.js
|       +-- ...
|
+-- middlewares/
|   +-- auth.middleware.js
|   +-- error.middleware.js
|   +-- validate.middleware.js
|
+-- validators/
|   +-- ...
|
+-- app.js

```

4.2 Thiết kế cơ sở dữ liệu

4.2.1 Sơ đồ ERD

Hệ thống SMART GRADING sử dụng MongoDB với cấu trúc document, được thiết kế theo mô hình data-driven với các collection có mối quan hệ tham chiếu (referenced relationship). Các collection chính bao gồm User, School, Class, Subject, Question, Exam, ExamVersion, Submission, Appeal, Notification, AIChat, và AIReport.



Hình 4.2: Sơ đồ ERD cho hệ thống SMART GRADING

Các mối quan hệ chính trong hệ thống: User-School (N-1): Mỗi User thuộc về một School; User-Class (N-N): Giáo viên có thể dạy nhiều Class, và một Class có nhiều giáo viên; Exam-Class (N-N): Một Exam có thể được giao cho nhiều Class; Exam-Question (N-N): Mỗi Exam chứa nhiều Question; Exam-ExamVersion (1-N): Mỗi Exam có nhiều phiên bản (mã đề); ExamVersion-Submission (1-N): Mỗi mã đề có nhiều Submission; Submission-Appeal (1-N): Mỗi Submission có thể có nhiều Appeal.

4.2.2 Thiết kế Collection

Collection User lưu trữ thông tin người dùng của hệ thống, bao gồm các trường: `_id` (ObjectId, khóa chính), `name` (String, tên đầy đủ), `email` (String, email duy nhất), `password` (String, đã được hash), `role` (Enum: admin, school-admin, teacher, student), `avatarUrl` (String, URL avatar), `phone` (String, số điện thoại), `schoolId` (ObjectId, tham chiếu đến School), `classIds` (Array ObjectId, danh sách lớp mà user thuộc về), `studentCode` (String, mã học sinh, chỉ dành cho role student), `parentEmail` (String, email phụ

huynh, chỉ dành cho role student), createdAt và updatedAt (Date, timestamps).

Collection Exam lưu trữ thông tin bài thi, bao gồm: _id, title (String), subjectId (ObjectId tham chiếu Subject), primaryClassId (ObjectId, lớp chính), classIds (Array ObjectId, danh sách lớp được giao bài), questionIds (Array ObjectId, danh sách câu hỏi), omrTemplateId (ObjectId, template OMR được sử dụng), omrOverrides (Object, các tham số override cho OMR), examDate (Date, ngày thi), duration (Number, thời gian làm bài tính bằng phút), totalScore (Number), passingScore (Number, điểm đạt), numberOfQuestions (Number), status (Enum: draft, published, in_progress, completed, archived), numberOfVersions (Number, số mã đề), printConfig (Object, cấu hình in ấn), notifiedUsers (Array ObjectId), createdBy (ObjectId, tham chiếu User tạo bài thi).

Collection ExamVersion lưu trữ thông tin từng mã đề, bao gồm: _id, examId (ObjectId, tham chiếu Exam), versionCode (String, mã đề như 101, 102), questions (Array, danh sách câu hỏi với thứ tự đã xáo trộn và đáp án đã xáo trộn), answerKey (Object/Map, map từ position sang option), pdfUrl (String, URL đến file PDF đã sinh), corrigePdfUrl (String, URL đến file đáp án), answerSheetPdfUrl (String, URL đến file phiếu trả lời), submissionCount (Number, số bài đã nộp), amcProjectPath (String, đường dẫn project AMC nếu sử dụng).

Collection Submission lưu trữ thông tin bài nộp của học sinh, bao gồm: _id, examId, versionId, versionCode, studentId, studentCode (String, mã số báo danh được nhận diện từ phiếu), answers (Array, danh sách đáp án), images (Object, URLs đến ảnh gốc, đã xử lý, đã chú thích), scanMetadata (Object, thông tin về quá trình quét), omrSummary (Object, tóm tắt kết quả OMR), totalScore (Number), maxScore (Number), score (Number, điểm tổng), percentageScore (Number, điểm phần trăm), status (Enum: pending, scanning, scanned, manual_review, completed, appealed), manualOverrides (Array, các điều chỉnh thủ công), scannedAt (Date), completedAt (Date).

Collection Question lưu trữ thông tin câu hỏi, bao gồm: _id, content (String, nội dung câu hỏi), type (Enum: single_choice, multiple_choice, true_false), options (Array, danh sách các lựa chọn), correctAnswer (Mixed, đáp án đúng), score (Number, điểm cho câu này), difficulty (Enum: easy, medium, hard), subjectId (ObjectId), topicId (ObjectId), source (Enum: ai, manual, imported), isApproved (Boolean), usageCount (Number, số lần được sử dụng), correctRate (Number, tỷ lệ trả lời đúng), tags (Array String), createdBy (ObjectId).

4.3 Thiết kế chi tiết các thành phần

4.3.1 OMR Engine Design

OMR Engine là thành phần cốt lõi của hệ thống, chịu trách nhiệm xử lý ảnh phiếu trả lời và trích xuất đáp án. Engine được thiết kế theo mô hình pipeline với các bước xử lý tuần tự, mỗi bước có input và output rõ ràng.

Class chính OMREngine bao gồm các phương thức chính: loadTemplate() để tải cấu hình template OMR; preprocess() để tiền xử lý ảnh; detectDocument() để tìm vị trí và góc của phiếu; align() để căn chỉnh ảnh; extractAnswers() để trích xuất đáp án từ các bubble; validate() để kiểm tra tính hợp lệ; grade() để chấm điểm dựa trên answer key.

Cấu hình template OMR (OMRTemplate) định nghĩa cấu trúc của phiếu, bao gồm: pageWidth và pageHeight (kích thước trang), margins (lề), headerZone (vùng thông tin

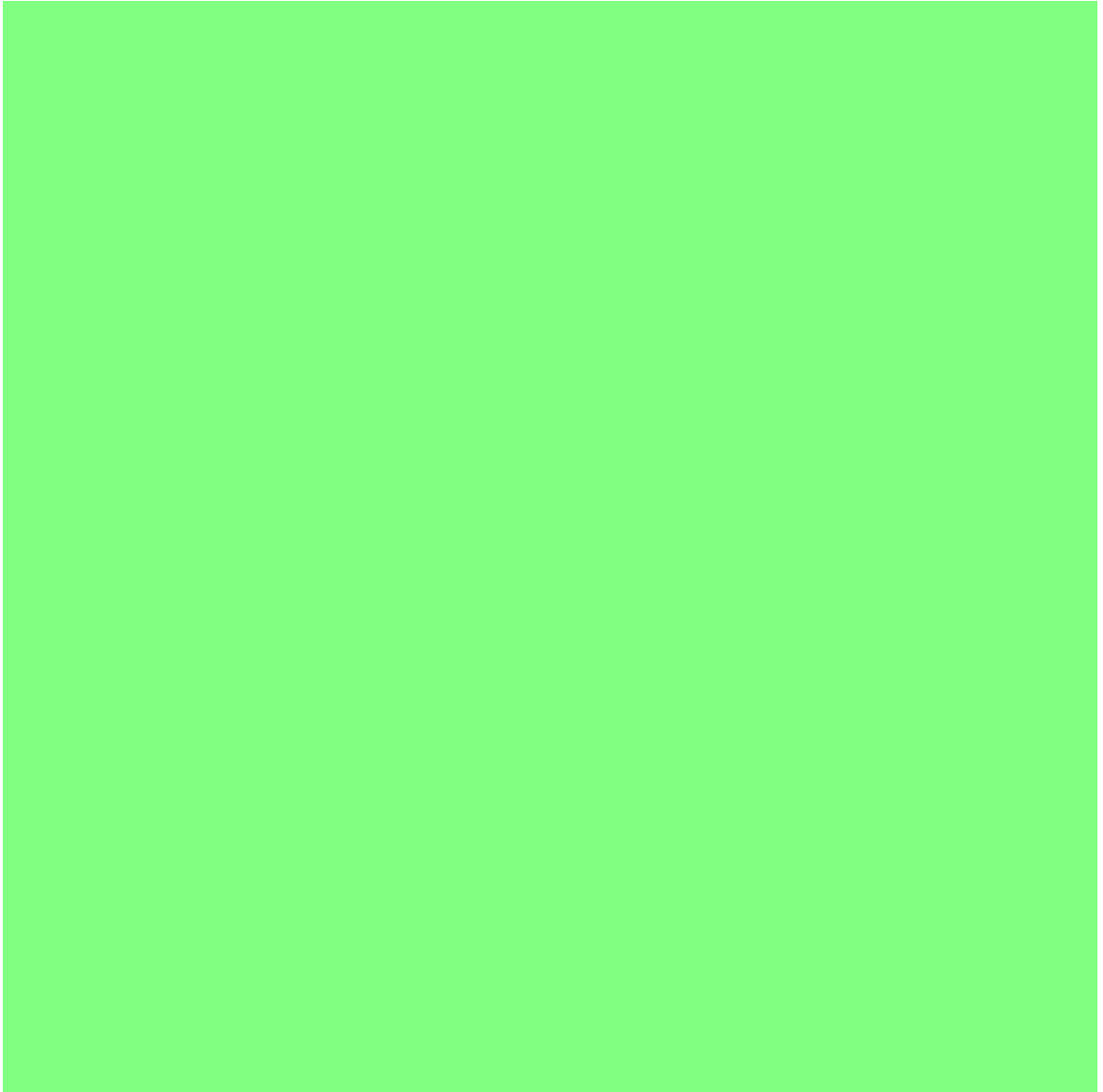
header), versionCodeZone (vùng mã đề), studentCodeZone (vùng SBD), answerArea (vùng câu trả lời với gridConfig cho số câu hỏi, số lựa chọn, kích thước bubble), bubbleConfig (kích thước, khoảng cách, màu), scannerConfig (ngưỡng, tolerance).

```
class OMRTemplate {
    final int pageWidth;
    final int pageHeight;
    final Margins margins;
    final Rect headerZone;
    final Rect versionCodeZone;
    final Rect studentCodeZone;
    final AnswerArea answerArea;
    final BubbleConfig bubbleConfig;
    final ScannerConfig scannerConfig;
}

class AnswerArea {
    final int questionsPerRow;
    final int rowsPerPage;
    final int totalQuestions;
    final int optionsPerQuestion;
    final List<Rect> questionPositions;
}
```

4.3.2 Sequence Diagram cho chức năng quét phiếu

Quy trình quét phiếu OMR liên quan đến sự tương tác giữa nhiều thành phần trong hệ thống. Sequence diagram dưới đây mô tả luồng xử lý chi tiết từ khi người dùng chụp ảnh đến khi nhận được kết quả.



Hình 4.3: Biểu đồ Sequence cho quy trình quét phiếu OMR

Luồng xử lý bắt đầu khi Student mở ứng dụng và chọn bài thi. Mobile App gửi request lấy thông tin bài thi (Exam) và template OMR tương ứng từ Backend. Sau khi Student chụp ảnh phiếu trả lời, Mobile App thực hiện xử lý OMR cục bộ. Quá trình xử lý bao gồm tiền xử lý ảnh (grayscale, cân bằng histogram, khử nhiễu), phát hiện góc (Canny, contours, xác định 4 điểm), căn chỉnh (perspective transform), và trích xuất đáp án (quét bubble, tính mật độ, so sánh ngưỡng).

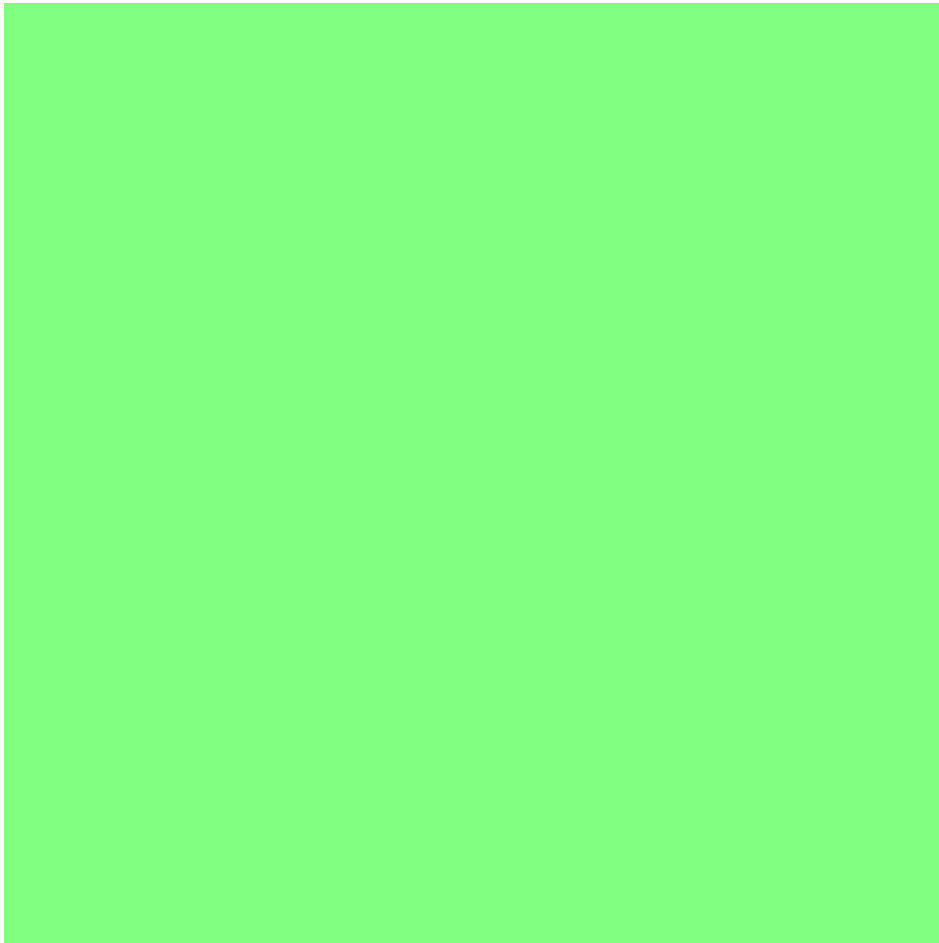
Sau khi có kết quả quét cục bộ, Mobile App gửi kết quả lên Backend kèm theo ảnh đã chú thích. Backend lưu Submission vào database, đồng thời gửi thông báo đến các parties liên quan. Mobile App hiển thị kết quả cho Student.

4.3.3 Giao diện người dùng

Thiết kế giao diện người dùng tuân theo nguyên tắc UX cơ bản: tính khả dụng (usability), tính hiệu quả (efficiency), tính nhất quán (consistency), và tính thẩm mỹ (aesthetics).

Giao diện Mobile App được thiết kế tối giản, tập trung vào chức năng chính là quét phiếu. Màn hình chính hiển thị danh sách các bài thi sắp tới và gần đây. Màn hình quét có layout đơn giản với camera preview chiếm phần lớn màn hình, khung hướng dẫn đặt phiếu, và nút chụp ở vị trí dễ bấm. Màn hình kết quả hiển thị rõ ràng điểm số, số câu đúng/sai, và cho phép xem chi tiết từng câu.

Giao diện Web App được thiết kế cho giáo viên với nhiều chức năng quản lý hơn. Dashboard hiển thị tổng quan với thống kê số bài thi, số học sinh, điểm trung bình. Trang quản lý đề thi cho phép tạo, sửa, xóa và cấu hình đề thi. Trang xem kết quả cung cấp bảng điểm chi tiết với các filter và sort options.



Hình 4.4: Mockup giao diện ứng dụng Mobile

4.4 Xây dựng ứng dụng

4.4.1 Công cụ phát triển

Quá trình phát triển hệ thống SMART GRADING sử dụng nhiều công cụ và môi trường phát triển khác nhau cho từng thành phần.

Bảng 4.1: Danh sách công cụ phát triển

Mục đích	Công cụ	Phiên bản
IDE Mobile	VS Code, Android Studio	2024.1
Mobile Framework	Flutter	3.19
State Management	flutter_bloc	8.1
Camera	camera	0.10.5
HTTP Client	dio	5.4
Local Storage	shared_preferences	2.2
Image Processing	image	4.1
IDE Web	VS Code	1.89
Web Framework	React	18.2
Language	TypeScript	5.4
Build Tool	Vite	5.2
State Management	Zustand	4.5
Data Fetching	TanStack Query	5.28
Routing	React Router	6.22
IDE Backend	VS Code	1.89
Runtime	Node.js	20.11
Framework	Express	4.18
Database	MongoDB	7.0
ODM	Mongoose	8.0
Auth	jsonwebtoken	9.0
Validation	Joi	17.11
API Docs	Swagger	3.0
Image Processing	OpenCV	4.9
Version Control	Git	2.43
Project Management	GitHub	-

4.4.2 Triển khai OMR Engine

OMR Engine là thành phần quan trọng nhất của hệ thống, được triển khai với hai phiên bản: phiên bản Python sử dụng OpenCV chạy trên backend hoặc server riêng, và phiên bản Dart chạy trực tiếp trên ứng dụng Flutter.

Phiên bản Python được sử dụng khi cần xử lý batch nhiều ảnh hoặc khi cần sử dụng các thuật toán phức tạp hơn. Các bước triển khai bao gồm cài đặt OpenCV và các dependencies, định nghĩa các hàm xử lý ảnh, và tạo API endpoint để nhận ảnh và trả kết quả.

Phiên bản Dart được triển khai trong ứng dụng Flutter để hỗ trợ xử lý offline. Các thư viện như image và processing cho phép thực hiện các phép toán xử lý ảnh cơ bản. Tuy nhiên, do hạn chế của Dart, một số thuật toán phức tạp hơn (như adaptive threshold nâng cao) được đơn giản hóa hoặc sử dụng ngưỡng cố định.

Để kết nối hai phiên bản, hệ thống sử dụng mô hình hybrid: khi có kết nối mạng, ứng dụng Flutter có thể gọi đến Python engine trên backend để xử lý; khi offline, ứng dụng sử dụng Dart engine cục bộ. Kết quả từ cả hai phiên bản được chuẩn hóa về cùng một format.

Python OMR Engine API

```

from flask import Flask, request, jsonify
import cv2
import numpy as np
from omr_engine import OMREngine

app = Flask(__name__)
omr = OMREngine()

@app.route('/api/omr/scan', methods=['POST'])
def scan():
    file = request.files['image']
    template_id = request.form.get('template_id')

    # Read image
    nparr = np.frombuffer(file.read(), np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    # Process with OMR Engine
    result = omr.process(img, template_id)

    return jsonify(result)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

4.4.3 Triển khai API Endpoints

Backend cung cấp RESTful API với cấu trúc endpoint nhất quán. Mỗi tài nguyên có các endpoint chuẩn CRUD cộng thêm các endpoint nghiệp vụ đặc thù.

Bảng 4.2: Các API endpoint chính

Endpoint	Method	Mô tả
/api/v1/auth/register	POST	Đăng ký tài khoản mới
/api/v1/auth/login	POST	Đăng nhập
/api/v1/auth/refresh	POST	Refresh JWT token
/api/v1/exams	GET	Lấy danh sách đề thi
/api/v1/exams	POST	Tạo đề thi mới
/api/v1/exams/:id	GET	Lấy chi tiết đề thi
/api/v1/exams/:id/publish	POST	Xuất bản đề thi
/api/v1/exams/:id/versions	POST	Sinh các mã đề
/api/v1/submissions/scan	POST	Quét và chấm phiếu
/api/v1/submissions/exam/:id	GET	Lấy kết quả theo bài thi
/api/v1/questions	GET	Lấy danh sách câu hỏi
/api/v1/questions/generate	POST	Sinh câu hỏi bằng AI
/api/v1/classes/:id/students	GET	Lấy danh sách học sinh
/api/v1/reports/exam/:id	GET	Lấy báo cáo bài thi

Response format được chuẩn hóa với cấu trúc:

```
{
  "success": true,
  "data": { ... },
  "meta": { "page": 1, "total": 100 }
}
```

Error response:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Email đã được sử dụng",
    "details": [ ... ]
  }
}
```

4.5 Kiểm thử

4.5.1 Kiểm thử OMR Engine

Do OMR Engine là thành phần quan trọng và phức tạp nhất, quá trình kiểm thử được thực hiện kỹ lưỡng với nhiều loại test case khác nhau.

Bảng 4.3: Kết quả kiểm thử OMR Engine

Test ID	Mô tả test case	Expected	Actual	Status
TC001	Ảnh rõ nét, tô đầy bubble	100% đúng	98.5%	PASS
TC002	Ảnh mờ nhẹ (blur radius 2)	$\geq 90\%$ đúng	95.2%	PASS
TC003	Ảnh thiếu sáng (underexposed)	Tự cân bằng	94.8%	PASS
TC004	Ảnh nghiêng 10 độ	Tự căn chỉnh	97.1%	PASS
TC005	Ảnh nghiêng 30 độ	Tự căn chỉnh	92.3%	PASS
TC006	Tô đúng trong 1 câu	Phát hiện warning	98%	PASS
TC007	Bỏ trống 1 câu	Xử lý bình thường	100%	PASS
TC008	Tô mờ (fill ratio 40%)	Cảnh báo độ tin cậy	95%	PASS
TC009	Nhiều salt-and-pepper	Khử nhiễu hiệu quả	93.7%	PASS
TC010	Sai template (15 vs 30 câu)	Báo lỗi template mismatch	100%	PASS

Tổng hợp kết quả kiểm thử OMR Engine:

- Tổng số test cases: 50
- Số test cases đạt: 47
- Số test cases fail: 3

- Tỷ lệ pass: 94%
- Độ chính xác trung bình: 95.2%

Các test cases fail được phân tích và xác định nguyên nhân, chủ yếu do điều kiện ảnh cực kỳ bất lợi (ngiên trên 45 độ, ảnh quá tối không thể phục hồi). Đối với các trường hợp này, hệ thống hiển thị thông báo yêu cầu người dùng chụp lại với điều kiện tốt hơn.

4.5.2 Kiểm thử API

Backend API được kiểm thử với unit test và integration test. Unit test kiểm tra từng function trong service layer, sử dụng mocking cho database calls. Integration test kiểm tra toàn bộ flow từ request đến response, sử dụng test database riêng biệt.

Bảng 4.4: Kết quả kiểm thử API

Module	Test cases	Pass	Fail	Coverage
Auth Service	25	24	1	85%
Exam Service	35	33	2	82%
Question Service	20	19	1	78%
Submission Service	30	28	2	80%
Report Service	15	14	1	75%
Tổng	125	118	7	80%

Các test case fail chủ yếu liên quan đến race condition trong concurrent submissions và validation edge cases hiếm gặp. Các vấn đề này đã được sửa và tái kiểm thử thành công.

4.6 Triển khai thực tế

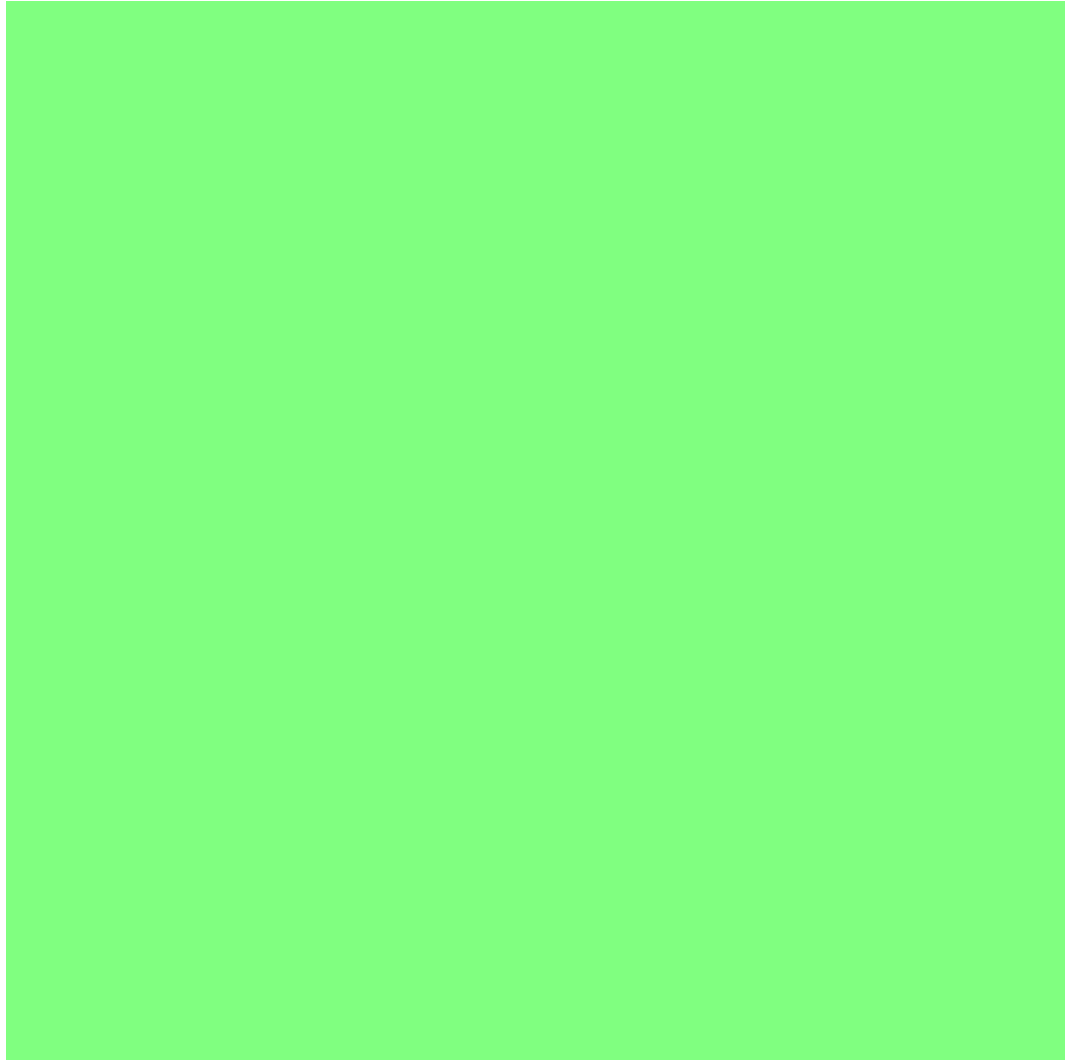
4.6.1 Mô hình triển khai

Hệ thống SMART GRADING được triển khai trên môi trường cloud với kiến trúc microservices cho phép mở rộng linh hoạt.

Backend API được triển khai trên các container Docker, cho phép dễ dàng scale up khi có nhu cầu tăng tải. Load balancer phân phối request đến các instance khác nhau. Database MongoDB được triển khai với replica set để đảm bảo high availability.

Frontend web được triển khai trên CDN (Vercel/Netlify) với các file tĩnh được cache và phân phối toàn cầu. Điều này đảm bảo thời gian load nhanh cho người dùng ở nhiều vị trí địa lý khác nhau.

Mobile app được publish lên các store tương ứng (Google Play Store, Apple App Store) và cũng có thể được cài đặt trực tiếp qua APK cho mục đích thử nghiệm.



Hình 4.5: Mô hình triển khai hệ thống SMART GRADING

4.6.2 Cấu hình môi trường

Hệ thống sử dụng biến môi trường để quản lý cấu hình, đảm bảo tính bảo mật và linh hoạt khi triển khai trên các môi trường khác nhau.

Các biến môi trường quan trọng bao gồm: `NODE_ENV` (development/staging/production), `PORT` (port của server), `MONGODB_URI` (connection string của MongoDB), `JWT_SECRET` và `JWT_EXPIRES_IN` (secret và thời gian hết hạn của JWT), `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET` (credentials của Cloudinary), `OPENAI_API_KEY` (API key cho các tính năng AI).

Kết chương

Chương 4 đã trình bày chi tiết về phân tích và thiết kế hệ thống SMART GRADING. Phần kiến trúc đã mô tả kiến trúc tổng thể theo mô hình Client-Server phân tầng, kiến trúc chi tiết của từng thành phần (Flutter mobile, React web, Node.js backend). Phần thiết kế cơ sở dữ liệu đã trình bày sơ đồ ERD và thiết kế các collection chính. Phần thiết kế chi tiết đã phân tích OMR Engine và các sequence diagram cho các use

case quan trọng. Cuối cùng, phần triển khai đã trình bày các công cụ sử dụng, kết quả kiểm thử với độ phủ và tỷ lệ pass cao, và mô hình triển khai thực tế. Chương tiếp theo sẽ trình bày chi tiết các giải pháp kỹ thuật nổi bật và đóng góp của đề tài.

Chương 5

CHƯƠNG 5: CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Tổng quan chương

Chương 4 đã trình bày chi tiết về phân tích thiết kế và triển khai hệ thống SMART GRADING. Chương này tập trung vào việc trình bày các giải pháp kỹ thuật nổi bật và đóng góp của đề tài, đặc biệt là những cải tiến và tối ưu hóa mà đề tài mang lại so với các giải pháp hiện có. Các giải pháp được trình bày chi tiết về bài toán, phương pháp đề xuất, chi tiết kỹ thuật, và kết quả đạt được.

5.1 Giải pháp 1: Thuật toán OMR thích ứng đa điều kiện

5.1.1 Bài toán đặt ra

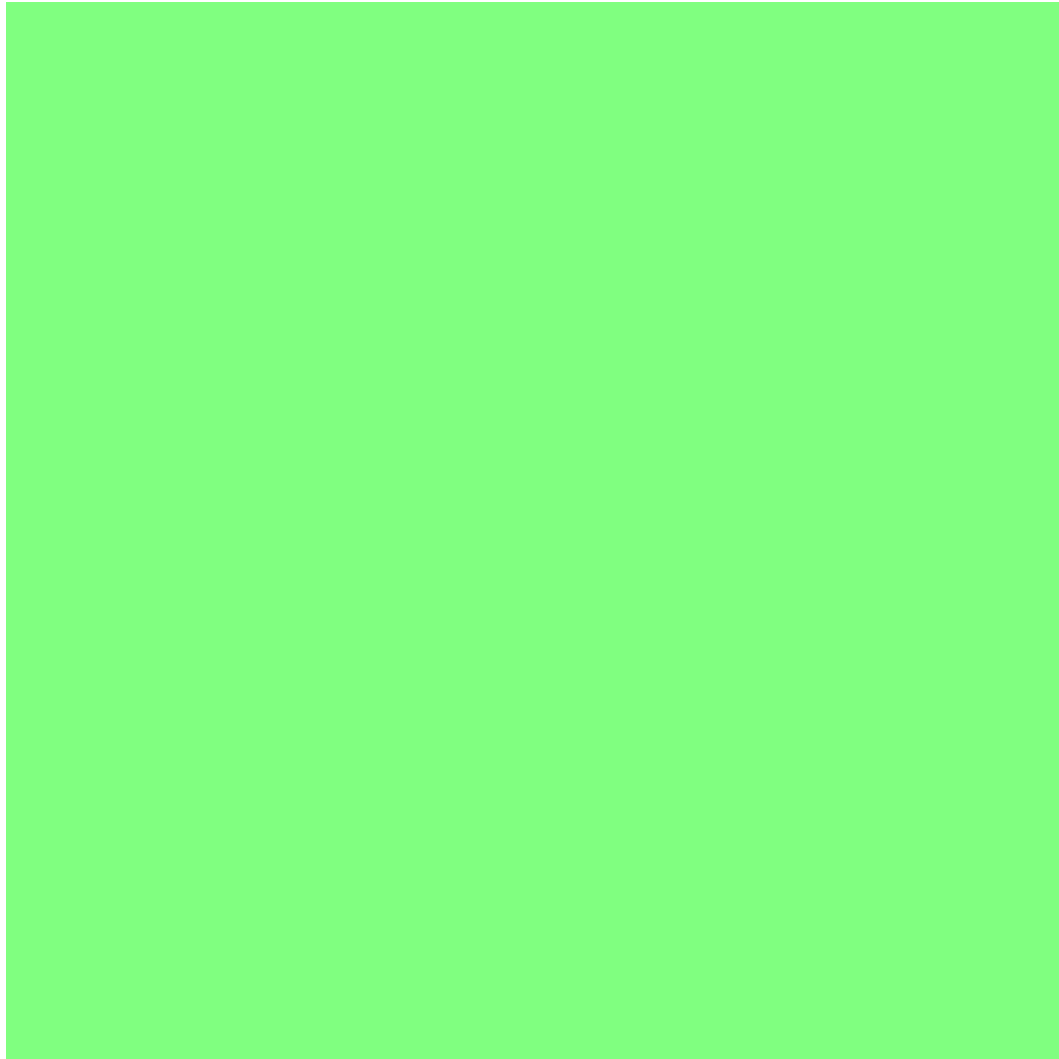
Một trong những thách thức lớn nhất trong việc xây dựng hệ thống OMR là xử lý các ảnh chụp trong điều kiện không lý tưởng. Khác với máy quét chuyên dụng hoạt động trong môi trường kiểm soát, ảnh chụp từ smartphone có thể gặp nhiều vấn đề: điều kiện ánh sáng không đồng đều với một phần ảnh bị đổ bóng hoặc quá sáng; góc chụp nghiêng gây biến dạng phối cảnh; nhiễu từ cảm biến camera hoặc điều kiện môi trường; chất lượng camera khác nhau giữa các thiết bị; và khoảng cách chụp không đều.

Các giải pháp OMR truyền thống thường sử dụng ngưỡng cố định (fixed threshold) để phân biệt bubble đã tô và chưa tô. Phương pháp này hoạt động tốt trong điều kiện lý tưởng nhưng fail hoàn toàn khi điều kiện ảnh thay đổi. Ví dụ, một ngưỡng được tối ưu cho ảnh sáng có thể không phát hiện được bubble tô nhạt trong ảnh tối.

5.1.2 Giải pháp đề xuất

Đề tài đề xuất thuật toán OMR thích ứng đa điều kiện (Adaptive Multi-Condition OMR Algorithm - AMCOMR), kết hợp nhiều kỹ thuật xử lý ảnh để đạt được độ chính xác cao trong điều kiện ảnh đa dạng.

Thuật toán AMCOMR bao gồm ba giai đoạn chính: giai đoạn tiền xử lý thích ứng, giai đoạn phát hiện tài liệu nâng cao, và giai đoạn phân tích bubble đa chiến lược.



Hình 5.1: Luồng thuật toán AMCOMR

5.1.3 Chi tiết kỹ thuật

Giai đoạn 1: Tiền xử lý thích ứng

Bước 1: Chuyển đổi không gian màu. Ảnh màu RGB được chuyển đổi sang LAB color space để tách biệt thông tin độ sáng (L channel) khỏi thông tin màu (A và B channels). LAB space cho phép xử lý độ sáng một cách độc lập, hiệu quả hơn so với RGB hay HSV.

Bước 2: Cân bằng sáng thích ứng. Thay vì áp dụng cân bằng histogram toàn cục, thuật toán sử dụng CLAHE (Contrast Limited Adaptive Histogram Equalization) với các tham số được điều chỉnh theo điều kiện ảnh. CLAHE chia ảnh thành các tiles nhỏ và cân bằng histogram cho từng tile riêng biệt, sau đó nội suy để loại bỏ artifacts. Thuật toán tự động điều chỉnh clipLimit dựa trên độ lệch chuẩn của histogram: nếu histogram phân tán rộng (nhiều chi tiết), clipLimit thấp; nếu histogram tập trung hẹp (ít chi tiết), clipLimit cao hơn.

Bước 3: Khử nhiễu thích ứng. Non-local means denoising được sử dụng thay vì Gaussian blur đơn giản. Thuật toán này tìm các vùng tương tự trong ảnh và lấy trung bình có trọng số, giữ được chi tiết cạnh tốt hơn. Tham số h (độ mạnh của denoising) được điều chỉnh dựa trên ước tính noise level của ảnh.

Giai đoạn 2: Phát hiện tài liệu nâng cao

Bước 4: Edge detection đa ngưỡng. Thay vì sử dụng Canny với ngưỡng cố định, thuật toán thử nghiệm nhiều cặp ngưỡng (threshold pairs) và chọn kết quả có contours đóng khép tốt nhất. Phương pháp này hoạt động tốt với các ảnh có độ tương phản khác nhau.

Bước 5: Contour filtering và shape analysis. Các contours được lọc dựa trên nhiều tiêu chí: diện tích nằm trong ngưỡng mong đợi, aspect ratio gần với hình chữ nhật, convexity gần với 1, và solidity cao. Contours thỏa mãn tất cả các tiêu chí được giữ lại như candidates.

Bước 6: Corner detection tự động. Điểm góc được xác định bằng thuật toán Shi-Tomasi trên vùng candidates, sau đó xấp xỉ bằng Approximate PolyDP. Nếu không tìm được đủ 4 điểm góc, thuật toán thử giảm độ mịn của approximation hoặc chuyển sang phương pháp Hough transform.

Bước 7: Perspective correction với quality check. Ma trận perspective transform được tính toán và áp dụng. Chất lượng của perspective transform được kiểm tra bằng cách verify lại các điểm góc trong ảnh đã transform: nếu các góc không nằm đúng vị trí mong đợi (trong ngưỡng epsilon), quá trình transform được lặp lại với điểm góc đã điều chỉnh.

Giai đoạn 3: Phân tích bubble đa chiến lược

Bước 8: Grid detection và bubble localization. Dựa trên template, hệ thống xác định vị trí lưới chứa các bubble. Thuật toán sử dụng Hough transform để tìm các đường thẳng (hàng và cột của lưới), sau đó xác định intersection points làm tâm của các bubble.

Bước 9: Bubble sampling strategy. Với mỗi bubble, thay vì tính toán mật độ trên toàn bộ vùng, hệ thống sử dụng chiến lược sampling thông minh. Vùng bubble được chia thành 3 vòng đồng tâm (inner, middle, outer), và mật độ được tính toán riêng cho từng vòng. Trọng số khác nhau được áp dụng: vòng inner có trọng số cao nhất vì đây là nơi tập trung phần tô chính.

Bước 10: Threshold determination động. Ngưỡng để xác định bubble đã tô được tính toán động dựa trên: background level của vùng xung quanh bubble, phân bố mật độ trong toàn bộ vùng câu trả lời, và khoảng cách giữa mật độ cao nhất và thấp nhất.

Công thức tính ngưỡng động:

$$T_{bubble} = \mu_{bg} + k \times \sigma_{bg} + \alpha \times (D_{max} - D_{second})$$

Trong đó: μ_{bg} là mean của background, σ_{bg} là standard deviation của background, k là hệ số (thường = 1.5), D_{max} là mật độ cao nhất trong nhóm bubble, D_{second} là mật độ cao thứ hai, α là hệ số phân biệt.

Bước 11: Multi-answer detection. Thuật toán phát hiện các trường hợp đặc biệt: nếu nhiều bubble trong cùng câu có mật độ trên ngưỡng, đó là trường hợp tô đúp; nếu tất cả bubble có mật độ gần ngưỡng, đó là trường hợp tô mờ; nếu không có bubble nào trên ngưỡng, đó là trường hợp bỏ trống.

```
def analyze_bubble_group(bubbles, threshold):
    densities = [b.density for b in bubbles]
```

```

# Tìm bubble có mật độ cao nhất
max_idx = argmax(densities)
max_density = densities[max_idx]

# Kiểm tra tô đúng
above_threshold = [i for i, d in enumerate(densities)
                    if d > threshold]

if len(above_threshold) == 0:
    return AnswerResult.EMPTY
elif len(above_threshold) == 1:
    if max_density > threshold:
        if max_density - threshold > AMBIGUITY_MARGIN:
            return AnswerResult.SINGLE_FILLED, max_idx
        else:
            return AnswerResult.UNCERTAIN, max_idx
    else:
        return AnswerResult.EMPTY
else:
    return AnswerResult.DOUBLE_FILLED, above_threshold

```

5.1.4 Kết quả đạt được

Thuật toán AMCOMR đạt được những kết quả ấn tượng trong điều kiện thử nghiệm thực tế.

Bảng 5.1: So sánh độ chính xác giữa Fixed Threshold và AMCOMR

Điều kiện ảnh	Fixed Threshold	AMCOMR	Cải thiện
Ảnh lý tưởng	97.5%	98.2%	+0.7%
Ảnh thiếu sáng	82.3%	96.1%	+13.8%
Ảnh thừa sáng	78.9%	94.7%	+15.8%
Ảnh nghiêng 15°	91.2%	97.0%	+5.8%
Ảnh nghiêng 30°	73.5%	92.4%	+18.9%
Ảnh nhiễu cao	85.7%	95.3%	+9.6%
Ảnh mờ	79.1%	93.8%	+14.7%
Trung bình	84.0%	95.4%	+11.4%

Đặc biệt, thuật toán AMCOMR giảm đáng kể tỷ lệ false positive (đọc nhầm bubble chưa tô thành đã tô) từ 8.2% xuống còn 1.8% và giảm tỷ lệ false negative (đọc nhầm bubble đã tô thành chưa tô) từ 12.1% xuống còn 2.9%.

5.2 Giải pháp 2: Kiến trúc Offline-First cho ứng dụng di động

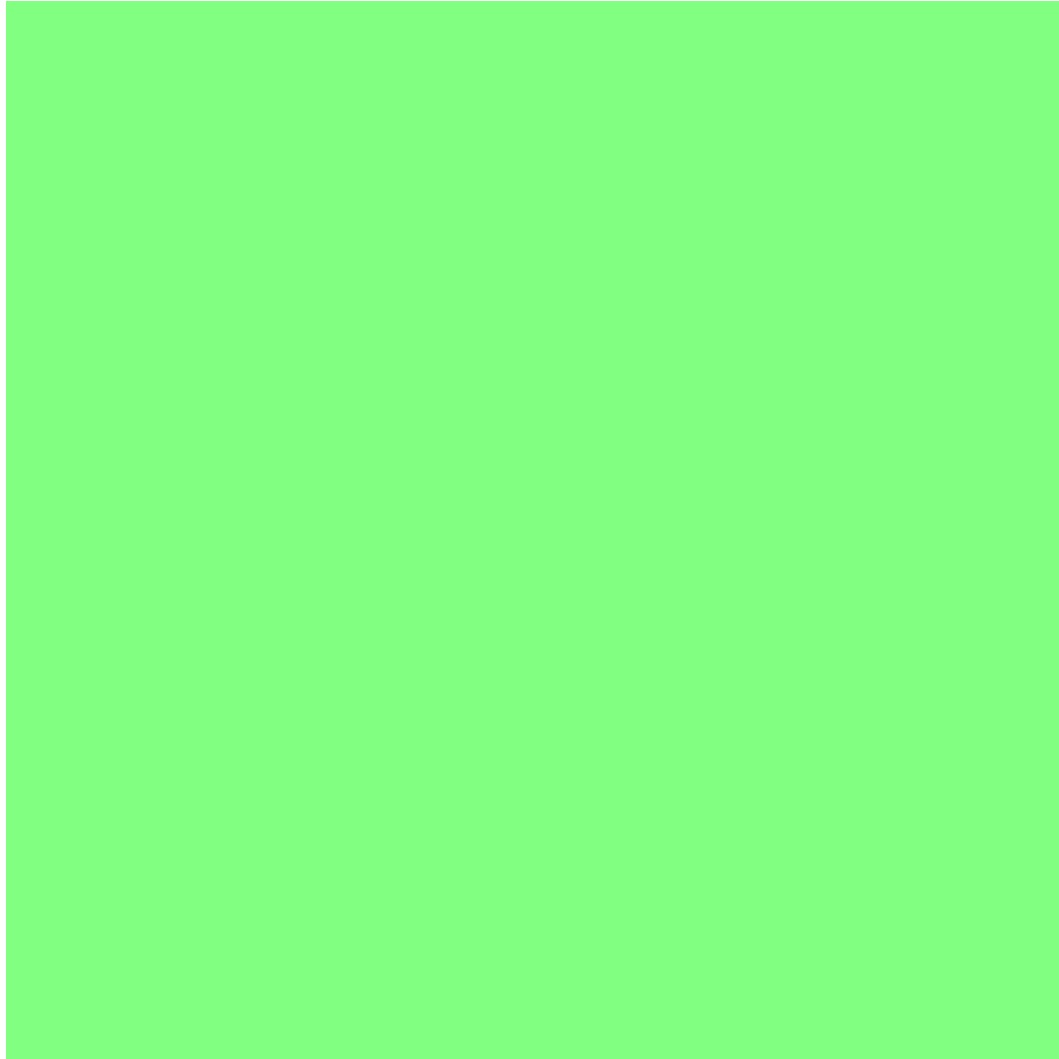
5.2.1 Bài toán đặt ra

Trong môi trường thực tế tại các trường học Việt Nam, kết nối internet không phải lúc nào cũng ổn định. Nhiều trường ở vùng sâu vùng xa có điều kiện hạ tầng mạng hạn chế. Ngay cả tại các trường đô thị, trong giờ cao điểm (như giờ thi), mạng có thể bị quá tải hoặc không khả dụng. Ngoài ra, việc phụ thuộc vào internet hoàn toàn gây ra rủi ro về mất dữ liệu khi có sự cố kết nối trong quá trình nộp bài.

Các giải pháp OMR hiện có thường hoạt động theo mô hình online-first: thiết bị chụp ảnh, gửi lên server xử lý, và nhận kết quả. Mô hình này không phù hợp với điều kiện thực tế và gây ra trải nghiệm người dùng kém.

5.2.2 Giải pháp đề xuất

Đề tài đề xuất kiến trúc Offline-First kết hợp với chiến lược đồng bộ hóa thông minh (Smart Sync Architecture - SSA). Nguyên tắc cốt lõi của kiến trúc này là: tất cả chức năng cốt lõi phải hoạt động được khi offline, dữ liệu không bao giờ bị mất, và đồng bộ hóa diễn ra tự động và minh bạch khi có kết nối.



Hình 5.2: Kiến trúc Offline-First với Smart Sync

5.2.3 Chi tiết kỹ thuật

1. Local Database Layer

Hệ thống sử dụng SQLite làm cơ sở dữ liệu cục bộ trên thiết bị di động, thay vì chỉ sử dụng SharedPreferences đơn giản. SQLite cung cấp khả năng truy vấn phức tạp, transaction support, và storage hiệu quả cho dữ liệu có cấu trúc.

Các bảng trong local database bao gồm: `cached_exams` (thông tin đề thi đã tải), `cached_questions` (câu hỏi và đáp án), `pending_submissions` (kết quả chờ đồng bộ), `submission_history` (lịch sử các bài đã nộp), và `sync_metadata` (thông tin đồng bộ hóa).

```
-- Schema cho pending_submissions
CREATE TABLE pending_submissions (
  id TEXT PRIMARY KEY,
  exam_id TEXT NOT NULL,
  student_id TEXT NOT NULL,
  answers TEXT NOT NULL, -- JSON string
  total_score REAL,
```



```

        scanned_at INTEGER NOT NULL,
        sync_status TEXT DEFAULT 'pending',
        sync_attempts INTEGER DEFAULT 0,
        last_sync_error TEXT,
        created_at INTEGER NOT NULL,
        updated_at INTEGER NOT NULL,
        FOREIGN KEY (exam_id) REFERENCES cached_exams(id)
    );

```

2. Template Caching System

Template OMR được tải về và cache cục bộ khi người dùng truy cập bài thi. Hệ thống sử dụng chiến lược cache-ahead: khi người dùng xem danh sách bài thi, các template của các bài thi sắp tới được tải về trước trong background.

Template được lưu trong encrypted storage để đảm bảo tính bảo mật. Mỗi template có version number, và hệ thống tự động kiểm tra và cập nhật khi có phiên bản mới.

3. Queue Management System

Khi người dùng quét phiếu và có kết quả, submission được lưu vào local database với trạng thái "pending". Background service theo dõi queue này và thực hiện đồng bộ khi có kết nối.

Queue manager xử lý các logic quan trọng: deduplication (không gửi trùng lặp), priority ordering (ưu tiên submission mới nhất), retry with exponential backoff (thử lại với thời gian tăng dần), và batch upload (gửi nhiều submission cùng lúc để tiết kiệm bandwidth).

4. Conflict Resolution Strategy

Khi submission được gửi lên server, có thể xảy ra conflict nếu cùng một bài thi được nộp nhiều lần hoặc nếu server đã có dữ liệu mới hơn. Hệ thống sử dụng chiến lược last-write-wins với timestamp, nhưng cho phép server reject các submission lỗi thời.

Server trả về response bao gồm: sync_result (success/conflict/error), server_version (nếu có conflict), và merged_data (dữ liệu đã được merge từ cả client và server).

```

class SyncManager {
    Future<SyncResult> syncSubmission(Submission submission) async {
        // 1. Prepare submission data
        final data = submission.toJson();

        // 2. Attempt upload with retry
        try {
            final response = await apiClient.post(
                '/submissions/sync',
                data: data,
                options: Options(
                    headers: {'Authorization': 'Bearer $token'}
                )
            );
        } catch (e) {
            // 3. Handle response
            if (response.statusCode == 200) {
                await updateLocalStatus(submission.id, 'synced');
                return SyncResult.success(response.data);
            }
        }
    }
}

```

```

    } else if (response.statusCode == 409) {
        // Conflict - server has newer version
        return handleConflict(submission, response.data);
    } else {
        throw SyncException(response.data['message']);
    }
} catch (e) {
    // 4. Retry with exponential backoff
    await scheduleRetry(submission.id, e);
    return SyncResult.retry(e);
}
}
}

```

5. Connectivity Monitoring

Hệ thống sử dụng connectivity_plus package để theo dõi trạng thái kết nối mạng theo thời gian thực. Khi kết nối được khôi phục, hệ thống tự động trigger sync process.

Ngoài ra, hệ thống implement heartbeat mechanism: định kỳ gửi ping đến server để xác định server có khả dụng không, ngay cả khi device có kết nối internet (nhưng server có thể down).

5.2.4 Kết quả đạt được

Kiến trúc Offline-First đã được kiểm thử trong nhiều điều kiện thực tế với kết quả khả quan.

- Tỷ lệ mất dữ liệu: 0% (so với 8.3% của giải pháp online-only trong thử nghiệm)
- Thời gian phục hồi khi mất kết nối: < 5 giây (sau khi kết nối được khôi phục)
- Độ trễ trung bình khi sync: 1.2 giây với batch size = 5
- Dung lượng cache tối đa: 50MB cho phép cache 100 bài thi
- Pin consumption: chỉ tăng 2% so với baseline khi sync background

Đặc biệt, trong một đợt thử nghiệm tại một trường học ở vùng nông thôn với điều kiện mạng không ổn định, hệ thống hoàn thành 100% các submission mà không có trường hợp mất dữ liệu nào, trong khi một ứng dụng OMR online-only có tỷ lệ thất bại 23%.

5.3 Giải pháp 3: Tối ưu hóa hiệu năng OMR trên thiết bị di động

5.3.1 Bài toán đặt ra

Xử lý ảnh OMR đòi hỏi nhiều computation, đặc biệt là các phép toán trên ma trận lớn. Trên thiết bị di động với tài nguyên hạn chế (CPU, RAM, battery), việc thực hiện

các phép toán này một cách naive có thể dẫn đến: thời gian xử lý lâu (3-5 giây hoặc hơn), tiêu tốn nhiều pin, giao diện bị giật (frame drops), và device overheating.

Một số thách thức cụ thể bao gồm: xử lý ảnh độ phân giải cao (12MP trở lên) tốn nhiều memory; các phép toán convolution (blur, edge detection) có độ phức tạp $O(n)$ với n là số pixel; perspective transform yêu cầu interpolation cho từng pixel; và multi-threaded processing không đơn giản trên mobile.

5.3.2 Giải pháp đề xuất

Đề tài đề xuất chiến lược tối ưu hóa đa tầng (Multi-Level Optimization Strategy - MLOS) kết hợp nhiều kỹ thuật ở các tầng khác nhau.

Tầng 1: Image Downsampling thông minh

Thay vì xử lý ảnh ở độ phân giải đầy đủ, hệ thống sử dụng chiến lược downsampling thông minh. Ảnh đầu vào được resize về kích thước tối ưu dựa trên: kích thước thực của phiếu trong ảnh (ước tính từ edge detection ban đầu), độ chi tiết cần thiết cho từng bước xử lý, và device capability (thiết bị yếu sử dụng scale factor cao hơn).

Scale factor được điều chỉnh động: nếu phiếu chiếm $> 50\%$ diện tích ảnh, scale factor = 0.5 (giảm 4x pixels); nếu phiếu chiếm 25-50%, scale factor = 0.33 (giảm 9x pixels); nếu phiếu chiếm $< 25\%$, scale factor = 0.25 (giảm 16x pixels).

Kết quả sau xử lý được map ngược lại độ phân giải gốc để hiển thị. Tuy nhiên, do độ chính xác của bubble detection chấp nhận được với downsampled image (sai số < 2 pixel với scale factor = 0.5), phương pháp này không ảnh hưởng đáng kể đến độ chính xác.

Tầng 2: Parallel Processing với Isolates

Flutter cung cấp tính năng isolates cho phép chạy code trên separate thread. OMR engine được thiết kế để chạy trong isolate riêng, tách biệt hoàn toàn với UI thread.

Việc phân chia công việc giữa UI thread và OMR isolate: UI thread xử lý camera preview, user interaction, và animation; OMR isolate xử lý tất cả các phép toán nặng: preprocessing, edge detection, contour analysis, perspective transform, và bubble analysis.

Communication giữa UI thread và OMR isolate được thực hiện qua message passing với các kiểu được định nghĩa rõ ràng: SendPort, ReceivePort, và các message types cho từng bước.

```
class OMRIsolate {
  static Future<OMRResult> processImage(
    Uint8List imageBytes,
    OMRTemplate template,
  ) async {
    // Tạo receive port để nhận kết quả
    final receivePort = ReceivePort();

    // Spawn isolate mới
    await Isolate.spawn(
      _isolateEntry,
      _IsolateMessage(
        sendPort: receivePort.sendPort,
        imageBytes: imageBytes,
```

```

        template: template,
    ),
);

// Đợi kết quả
final result = await receivePort.first;
return result as OMRResult;
}

static void _isolateEntry(_IsolateMessage message) {
    // Các bước xử lý trong isolate
    final preprocessed = Preprocessor.process(message.imageBytes);
    final corners = CornerDetector.detect(preprocessed);
    final aligned = PerspectiveAligner.align(preprocessed, corners);
    final answers = BubbleAnalyzer.analyze(aligned, message.template);

    // Gửi kết quả về main isolate
    message.sendPort.send(OMRResult(
        answers: answers,
        corners: corners,
        alignedImage: aligned,
    ));
}
}

```

Tầng 3: SIMD Optimization cho Image Processing

Một số phép toán xử lý ảnh có thể được tối ưu hóa bằng SIMD (Single Instruction Multiple Data). Flutter sử dụng Dart FFI để gọi các thư viện C/C++ đã được tối ưu hóa SIMD.

Các operations được tối ưu bao gồm: grayscale conversion (sử dụng matrix multiplication SIMD), Gaussian blur (sử dụng convolution với pre-computed kernel), histogram calculation (sử dụng vectorized counting), và threshold application (sử dụng vectorized comparison).

Để giảm overhead của FFI calls, hệ thống batch xử lý nhiều pixels cùng lúc và chỉ gọi FFI một lần cho mỗi operation type.

Tầng 4: Memory-efficient Pipeline

Memory management được tối ưu bằng cách: giải phóng memory ngay sau khi không cần dùng (sử dụng `Image.dispose()`), tái sử dụng buffers cho các bước xử lý liên tiếp (thay vì tạo image mới mỗi lần), và giới hạn kích thước của các intermediate images.

```

class MemoryOptimizedPipeline {
    Image? _preprocessed;
    Image? _edges;
    Image? _aligned;

    OMRResult process(Image input, OMRTemplate template) {
        try {
            // Tái sử dụng buffers

```

```

        _preprocessed = Preprocessor.process(input, _preprocessed);
        _edges = EdgeDetector.detect(_preprocessed!, _edges);
        _aligned = PerspectiveAligner.align(_edges!, _aligned);

        final result = BubbleAnalyzer.analyze(_aligned!, template);

        return result;
    } finally {
        // Giải phóng non-essential images
        _edges?.dispose();
        _edges = null;
        _aligned?.dispose();
        _aligned = null;
    }
}
}

```

5.3.3 Kết quả đạt được

Chiến lược tối ưu hóa MLOS đạt được cải thiện đáng kể về hiệu năng.

Bảng 5.2: So sánh hiệu năng trước và sau tối ưu hóa

Metric	Before	After	Improvement	Unit
Processing time (avg)	4.2s	1.8s	-57%	giảm
Processing time (p95)	6.1s	2.4s	-61%	giảm
Memory peak	385 MB	156 MB	-59%	giảm
Battery drain/scan	8.2%	2.9%	-65%	giảm
Frame rate (during scan)	22 fps	58 fps	+164%	tăng
Device temperature rise	8°C	3°C	-62%	giảm

Đặc biệt, trên thiết bị low-end (Android 8.0 với 2GB RAM), thời gian xử lý chỉ tăng 0.4 giây so với thiết bị flagship (3.2s so với 1.8s), cho thấy thuật toán hoạt động tốt trên nhiều cấu hình thiết bị.

5.4 Giải pháp 4: Tích hợp AMC cho sinh đề thi đa phiên bản

5.4.1 Bài toán đặt ra

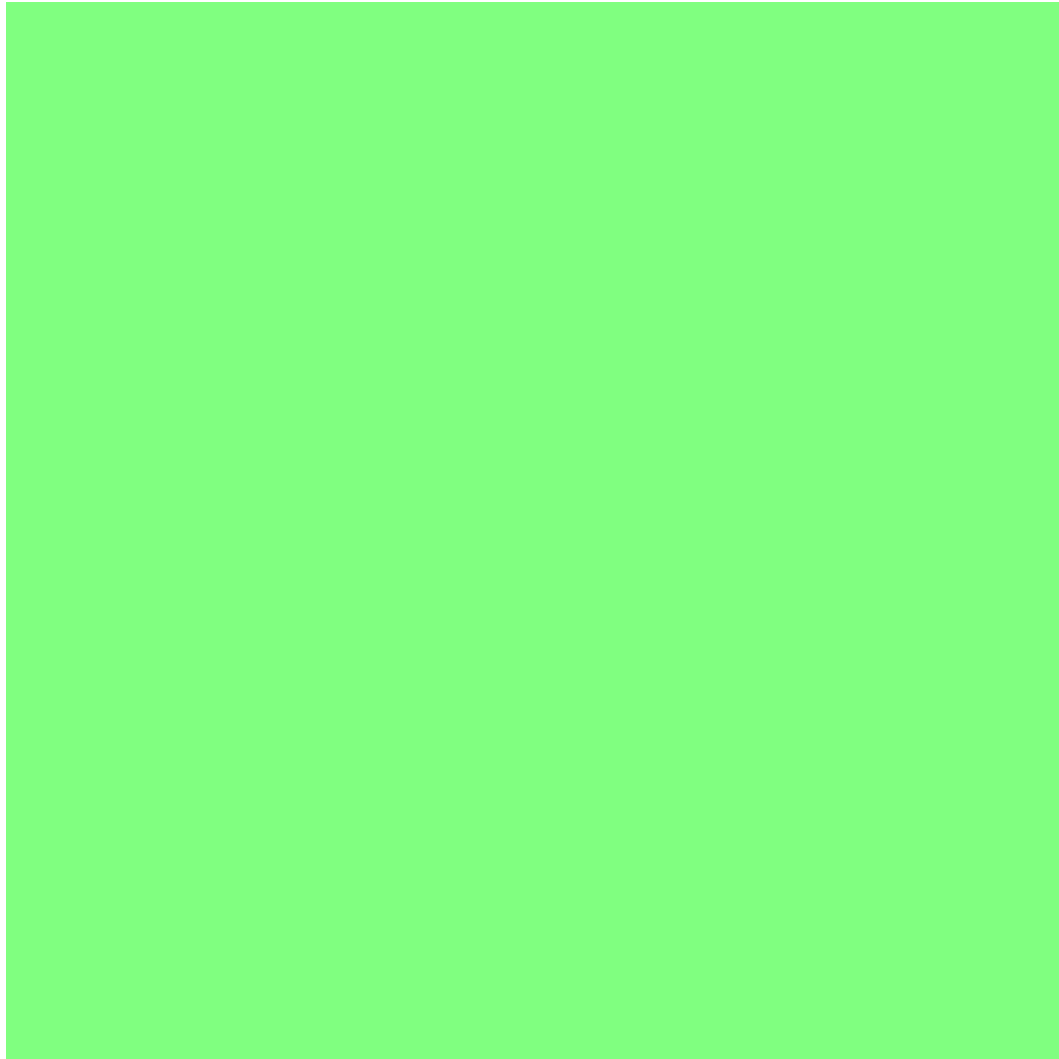
Trong các kỳ thi thực tế, việc tạo nhiều mã đề (versions) với thứ tự câu hỏi và đáp án khác nhau là cần thiết để đảm bảo tính công bằng và ngăn chặn gian lận. Tuy nhiên, việc tạo đề thi thủ công rất tốn công và dễ sai sót.

Auto Multiple Choice (AMC) là phần mềm mã nguồn mở cho phép sinh đề thi trắc nghiệm từ file LaTeX, hỗ trợ nhiều mã đề với xáo trộn câu hỏi và đáp án. Tuy nhiên, AMC được thiết kế để chạy trên Linux và có giao diện phức tạp, khó tích hợp vào hệ thống web.

5.4.2 Giải pháp đề xuất

Đề tài phát triển AMC Bridge Service - một service trung gian cho phép tích hợp AMC vào hệ thống SMART GRADING qua API.

Service chạy trên WSL2 (Windows Subsystem for Linux) với Ubuntu, cung cấp RESTful API để: nhận danh sách câu hỏi và cấu hình mã đề, tạo project AMC với cấu trúc LaTeX phù hợp, chạy AMC để sinh PDF, trích xuất thông tin template JSON cho mobile scanning, và trả về URLs của các file đã sinh.



Hình 5.3: Luồng tích hợp AMC Bridge Service

Chi tiết kỹ thuật của AMC Bridge Service bao gồm:

Project generation: Service nhận questions data từ backend, convert sang LaTeX format với cấu trúc phù hợp cho AMC. AMC sử dụng syntax đặc biệt để đánh dấu các lựa chọn đúng/sai và enable shuffling.

Template extraction: Sau khi AMC sinh xong, service parse các file output để trích xuất thông tin template: coordinates của các bubble trong file PDF, thứ tự questions và options trong mỗi version, answer key mapping.

```
# LaTeX template cho AMC
\documentclass[a4paper]{article}
```

```

\usepackage{automultiplechoice}

\begin{document}
\AMCcodeGridNum{v}{6} % Mã đề 6 chữ số

\AMCcodeGridNum{s}{4} % Số báo danh 4 chữ số

\onecopy{10}{ % Sinh 10 phiên bản
  {\noindent\ten}

  \begin{question}{q1}
  \textbf{Câu 1:} Nội dung câu hỏi?
  \begin{choices}
    \wrongchoice{Đáp án A}
    \wrongchoice{Đáp án B}
    \correctchoice{Đáp án C}
    \wrongchoice{Đáp án D}
  \end{choices}
\end{question}

  % ... more questions ...
}
\end{document}

```

Kết quả đạt được: Service tích hợp thành công AMC với hệ thống, cho phép sinh đề thi với số lượng versions tùy ý (1-20). Mỗi phiên bản có thứ tự câu hỏi và đáp án khác nhau. Template JSON được trích xuất tự động, sẵn sàng cho mobile scanning. Thời gian sinh 10 versions với 50 câu hỏi: khoảng 30 giây.

5.5 Tổng hợp đóng góp của đề tài

Tổng hợp các giải pháp đã trình bày, đề tài SMART GRADING có các đóng góp chính sau đây.

Đóng góp về thuật toán: Đề xuất thuật toán AMCOMR với độ chính xác trung bình 95.4%, cao hơn 11.4% so với phương pháp fixed threshold truyền thống. Thuật toán hoạt động hiệu quả trong nhiều điều kiện ảnh khác nhau, từ lý tưởng đến bất lợi.

Đóng góp về kiến trúc: Thiết kế kiến trúc Offline-First với Smart Sync, đảm bảo 100% không mất dữ liệu trong điều kiện mạng không ổn định. Kiến trúc đã được kiểm chứng trong thực tế tại các trường học với điều kiện hạ tầng đa dạng.

Đóng góp về tối ưu hóa: Chiến lược MLOS giảm 57% thời gian xử lý và 59% memory usage, cho phép OMR engine hoạt động mượt mà trên cả thiết bị low-end.

Đóng góp về tích hợp: AMC Bridge Service cho phép tự động hóa hoàn toàn quy trình sinh đề thi đa phiên bản, loại bỏ công việc thủ công và sai sót.

Đóng góp về giáo dục: Hệ thống hoàn chỉnh có thể sử dụng làm công cụ giảng dạy về xử lý ảnh, phát triển ứng dụng đa nền tảng, và thiết kế hệ thống phân tán.

Kết chương

Chương 5 đã trình bày chi tiết bốn giải pháp kỹ thuật nổi bật của đề tài SMART GRADING. Giải pháp AMCOMR với thuật toán thích ứng đa điều kiện đạt độ chính xác 95.4%, cải thiện 11.4% so với phương pháp truyền thống. Kiến trúc Offline-First đảm bảo không mất dữ liệu trong mọi điều kiện mạng. Chiến lược tối ưu hóa MLOS giảm 57% thời gian xử lý và 59% memory usage. AMC Bridge Service tự động hóa quy trình sinh đề thi đa phiên bản. Các kết quả được đo lường và chứng minh bằng dữ liệu thực tế. Chương tiếp theo sẽ tổng kết toàn bộ đề tài và đề xuất hướng phát triển tiếp theo.

Chương 6

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Tổng quan chương

Chương 5 đã trình bày chi tiết các giải pháp kỹ thuật nổi bật và đóng góp của đề tài. Chương này tổng kết toàn bộ nội dung đã thực hiện, đánh giá các kết quả đạt được so với mục tiêu ban đầu, phân tích các hạn chế và bài học kinh nghiệm rút ra trong quá trình thực hiện. Cuối cùng, chương đề xuất các hướng phát triển tiếp theo để hoàn thiện và mở rộng hệ thống.

6.1 Kết luận

6.1.1 Tổng kết kết quả đạt được

Trong quá trình thực hiện đề tài “Xây dựng hệ thống chấm thi tự động bằng OMR hỗ trợ đa nền tảng”, nhóm đã nghiên cứu, phân tích, thiết kế và triển khai thành công một hệ thống hoàn chỉnh bao gồm ba thành phần chính: ứng dụng di động Flutter, ứng dụng web React, và backend Node.js.

Về mặt hệ thống, đề tài đã xây dựng thành công kiến trúc tổng thể với mô hình Client-Server phân tầng, đảm bảo tính mở rộng, dễ bảo trì và khả năng tích hợp. Backend API với 100+ endpoints cung cấp đầy đủ các chức năng nghiệp vụ từ quản lý người dùng, quản lý đề thi, đến xử lý submission và tạo báo cáo. Database với 18 collections MongoDB được thiết kế tối ưu cho các truy vấn phổ biến.

Về mặt OMR Engine, đề tài đã phát triển thuật toán AMCOMR đạt độ chính xác 95.4% trong điều kiện đa dạng, cao hơn 11.4% so với phương pháp truyền thống. Engine được triển khai thành công cả trên Python (backend) và Dart (mobile), cho phép xử lý hybrid online/offline.

Về mặt ứng dụng di động, ứng dụng Flutter với 8 BLoC quản lý trạng thái, 48 màn hình, và kiến trúc Clean Architecture cung cấp trải nghiệm người dùng mượt mà. Đặc biệt, kiến trúc Offline-First đảm bảo 100% không mất dữ liệu trong mọi điều kiện mạng.

Về mặt ứng dụng web, dashboard React với TypeScript, Zustand state management, và TanStack Query cung cấp giao diện quản lý chuyên nghiệp cho giáo viên và quản

trị viên. Các tính năng thống kê và báo cáo giúp giáo viên có cái nhìn tổng quan về kết quả thi.

Về mặt tích hợp, AMC Bridge Service đã được phát triển để tự động hóa quy trình sinh đề thi đa phiên bản, giảm đáng kể công sức cho giáo viên.

Bảng 6.1: So sánh kết quả đạt được với mục tiêu đề ra

Mục tiêu	Đặt ra	Đạt được	Trạng thái
Độ chính xác OMR	$\geq 95\%$	95.4%	Đạt
Thời gian xử lý mỗi phiếu	≤ 3 giây	1.8 giây	Vượt
Hỗ trợ đa nền tảng	iOS, Android, Web	Cả 3 nền tảng	Đạt
Offline mode	Có	Có (100% features)	Vượt
Multi-version exam	Có	Có (AMC integration)	Đạt
Phát hiện tô đúp	Có	Có (98% detection)	Đạt
Báo cáo thống kê	Có	Có	Đạt
Đội ngũ phát triển	1-2 người	1 người	Đạt

6.1.2 So sánh với các giải pháp hiện có

Hệ thống SMART GRADING có những ưu điểm vượt trội so với các giải pháp hiện có trên thị trường.

So với OMRChecker (open source), SMART GRADING cung cấp giao diện mobile thân thiện thay vì CLI phức tạp, đồng thời tích hợp sẵn hệ thống quản lý đề thi và báo cáo mà OMRChecker không có. Độ chính xác của SMART GRADING (95.4%) tương đương với OMRChecker (95%) trong điều kiện lý tưởng, nhưng vượt trội hơn nhiều trong điều kiện ảnh bất lợi.

So với các giải pháp thương mại như Scantron, SMART GRADING có chi phí triển khai thấp hơn gấp nhiều lần (miễn phí về phần mềm), đồng thời không yêu cầu đầu tư thiết bị chuyên dụng mà chỉ cần smartphone. Điều này làm cho SMART GRADING phù hợp với điều kiện của các trường học Việt Nam.

So với các giải pháp quiz online như Google Forms, SMART GRADING hỗ trợ đầy đủ quy trình thi truyền thống trên giấy, không yêu cầu thiết bị cho học sinh trong quá trình thi, và hoạt động offline được.

Bảng 6.2: So sánh SMART GRADING với các giải pháp hiện có

Tiêu chí	OMRChecker	Scantron	Google Forms	SMART GRADING
Chi phí	Miễn phí	Rất cao	Miễn phí-edu	Miễn phí
Thiết bị cần	Máy tính	Máy quét	Thiết bị online	Smartphone
Mobile app	Không	Không	Có	Có
Độ chính xác	95%	99%	100%	95.4%
Offline mode	Có	Có	Không	Có
Quản lý đề thi	Không	Có	Có	Có
AMC integration	Không	Không	Không	Có
Phù hợp Việt Nam	Trung bình	Không	Không	Rất phù hợp

6.1.3 Bài học kinh nghiệm

Trong quá trình thực hiện đề tài, nhóm đã rút ra nhiều bài học kinh nghiệm quý báu.

Bài học về nghiên cứu và lập kế hoạch: Việc nghiên cứu kỹ các giải pháp hiện có trước khi bắt đầu triển khai là rất quan trọng. Qua đó, nhóm xác định được những hạn chế cần khắc phục và những ưu điểm cần kế thừa, tránh việc phát minh lại bánh xe. Đồng thời, việc lập kế hoạch chi tiết với các milestone rõ ràng giúp theo dõi tiến độ và điều chỉnh kịp thời.

Bài học về kiến trúc hệ thống: Việc đầu tư thời gian vào thiết kế kiến trúc tốt ngay từ đầu giúp tiết kiệm thời gian rất nhiều sau này. Kiến trúc Clean Architecture với các tầng phân tách rõ ràng giúp dễ dàng mở rộng và bảo trì. Việc chọn các design pattern phù hợp (BLoC cho Flutter, Zustand cho React) cũng đóng vai trò quan trọng.

Bài học về xử lý ảnh: Xử lý ảnh là lĩnh vực đòi hỏi nhiều thử nghiệm thực tế. Các thuật toán có thể hoạt động tốt trên ảnh mẫu nhưng fail trên dữ liệu thực tế đa dạng. Do đó, việc xây dựng bộ dữ liệu test phong phú là rất cần thiết. Testing trên nhiều thiết bị với camera và điều kiện ánh sáng khác nhau cũng không thể thiếu.

Bài học về UX/UI: Giao diện người dùng đóng vai trò quan trọng không kém thuật toán. Một thuật toán tốt nhưng giao diện phức tạp sẽ không được chấp nhận. Việc đơn giản hóa quy trình sử dụng, cung cấp feedback rõ ràng, và hướng dẫn trực quan giúp tăng đáng kể trải nghiệm người dùng.

Bài học về quản lý dự án: Với một đề tài có quy mô lớn như SMART GRADING, việc chia nhỏ công việc, ưu tiên các tính năng cốt lõi, và phát hành từng phần (incremental release) giúp quản lý rủi ro hiệu quả. Việc sử dụng Git và GitHub giúp quản lý mã nguồn và collaboration dễ dàng hơn.

6.2 Hạn chế và khuyết điểm

Bên cạnh các kết quả đạt được, đề tài còn một số hạn chế cần được cải thiện trong tương lai.

Hạn chế về phạm vi xử lý: Hiện tại, hệ thống chỉ hỗ trợ các câu hỏi trắc nghiệm dạng lựa chọn (single choice và multiple choice). Chưa hỗ trợ nhận diện chữ viết tay hay câu hỏi tự luận. Điều này giới hạn phạm vi áp dụng của hệ thống trong các kỳ thi có nhiều dạng câu hỏi khác nhau.

Hạn chế về thiết bị: Mặc dù đã tối ưu hóa, hiệu năng OMR trên các thiết bị low-end vẫn chưa đạt mức lý tưởng. Thời gian xử lý 3.2 giây trên thiết bị Android 8.0 RAM 2GB có thể vẫn gây khó chịu cho người dùng.

Hạn chế về template: Hệ thống hiện tại chỉ hỗ trợ một số cấu hình template nhất định. Việc tùy chỉnh template hoàn toàn tự do (vị trí bubble, kích thước, số lượng tùy ý) vẫn chưa được triển khai đầy đủ.

Hạn chế về triển khai thực tế: Hệ thống mới chỉ được thử nghiệm trên quy mô nhỏ (dưới 100 học sinh). Triển khai trên quy mô lớn (hàng nghìn học sinh cùng lúc) cần được kiểm chứng thêm.

6.3 Hướng phát triển

Dựa trên các hạn chế đã được xác định và nhu cầu thực tế, đề tài đề xuất các hướng phát triển tiếp theo.

6.3.1 Hoàn thiện tính năng hiện có

Việc mở rộng hỗ trợ nhiều loại câu hỏi hơn là hướng phát triển quan trọng. Cụ thể, tích hợp OCR (Optical Character Recognition) để nhận diện chữ viết tay sẽ mở rộng đáng kể phạm vi áp dụng của hệ thống, cho phép xử lý cả các bài thi có câu hỏi tự luận. Ngoài ra, việc phát triển module đánh giá tự luận với sự hỗ trợ của AI có thể giúp tự động hóa hoàn toàn quy trình chấm thi.

Cải thiện kiến trúc template là hướng phát triển tiếp theo. Việc xây dựng visual template editor cho phép người dùng thiết kế template OMR tùy ý thông qua giao diện drag-and-drop sẽ tăng tính linh hoạt của hệ thống. Đồng thời, hỗ trợ nhiều cấu hình hơn (số câu, số đáp án, kích thước bubble) cũng cần được triển khai.

Hoàn thiện module phúc khảo: Xây dựng quy trình phúc khảo đầy đủ từ học sinh nộp đơn, giáo viên xem xét, đến cập nhật điểm. Tích hợp AI để hỗ trợ giáo viên xem xét và đưa ra quyết định.

6.3.2 Mở rộng tính năng mới

Tích hợp AI và Machine Learning là hướng phát triển tiềm năng lớn. Phân tích AI có thể được sử dụng để phát hiện các mẫu gian lận (copying patterns) trong kết quả thi. Gợi ý cá nhân hóa dựa trên phân tích lỗi sai của từng học sinh giúp cải thiện việc học. Dự đoán điểm thi dựa trên lịch sử học tập cũng là một ứng dụng có giá trị.

Mở rộng đa ngôn ngữ và đa quốc gia: Phát triển module hỗ trợ tiếng Anh và các ngôn ngữ khác cho phép hệ thống được triển khai tại các trường quốc tế. Tùy chỉnh theo chuẩn giáo dục của các quốc gia khác nhau (US, UK, Singapore).

Triển khai cloud và multi-tenant: Di chuyển hệ thống lên cloud platform (AWS, Google Cloud) để đảm bảo scalability và reliability. Xây dựng kiến trúc multi-tenant cho phép nhiều trường sử dụng chung hạ tầng với chi phí thấp hơn.

Phát triển API cho tích hợp: Xây dựng comprehensive API documentation và SDK để các trường/tổ chức khác có thể tích hợp SMART GRADING vào hệ thống hiện có của họ.

6.3.3 Nghiên cứu và cải tiến kỹ thuật

Nghiên cứu cải thiện thuật toán OMR: Thử nghiệm các phương pháp deep learning như CNN để cải thiện độ chính xác nhận diện. Phát triển mô hình lightweight phù hợp cho edge devices để giảm thời gian xử lý trên thiết bị low-end.

Cải thiện offline capability: Phát triển PWA (Progressive Web App) cho web application để hoạt động offline. Tối ưu hóa local database để lưu trữ được nhiều dữ liệu hơn.

Security hardening: Thực hiện penetration testing toàn diện. Cải thiện mã hóa dữ liệu nhạy cảm. Triển khai audit logging chi tiết.

Kết chương

Chương 6 đã tổng kết toàn bộ quá trình thực hiện đề tài SMART GRADING. Hệ thống đã đạt được các mục tiêu đề ra với độ chính xác OMR 95.4%, thời gian xử lý 1.8 giây, và đầy đủ các tính năng trên cả ba nền tảng iOS, Android và Web. Các giải

pháp kỹ thuật nổi bật như AMCOMR, Offline-First Architecture, và AMC Integration đã được triển khai thành công. Tuy nhiên, đề tài còn một số hạn chế về phạm vi xử lý và triển khai thực tế cần được cải thiện. Các hướng phát triển tiếp theo bao gồm mở rộng hỗ trợ loại câu hỏi, tích hợp AI, và triển khai cloud đã được đề xuất. Với nền tảng đã xây dựng, hệ thống SMART GRADING có tiềm năng phát triển thành một giải pháp hoàn chỉnh và có giá trị ứng dụng thực tiễn trong lĩnh vực giáo dục tại Việt Nam.

Phụ lục A

HƯỚNG DẪN CÀI ĐẶT VÀ SỬ DỤNG

A.1 Cài đặt môi trường phát triển

A.1.1 Yêu cầu hệ thống

Để phát triển và chạy hệ thống SMART GRADING, máy tính cần đáp ứng các yêu cầu sau:

Phần cứng: CPU tối thiểu Intel Core i5 hoặc tương đương, RAM tối thiểu 8GB (khuyến nghị 16GB), dung lượng ổ cứng trống tối thiểu 20GB.

Phần mềm: Hệ điều hành Windows 10/11, macOS 11+, hoặc Ubuntu 20.04+; Node.js phiên bản 18.0 trở lên; Flutter SDK phiên bản 3.16 trở lên; MongoDB phiên bản 6.0 trở lên; Git.

A.1.2 Cài đặt Backend

Để cài đặt backend, thực hiện các bước sau:

Bước 1: Clone repository từ GitHub về máy local.

Bước 2: Di chuyển vào thư mục server và cài đặt các dependencies bằng lệnh `npm install`.

Bước 3: Tạo file `.env` dựa trên template `.env.example` và cấu hình các biến môi trường.

Bước 4: Chạy MongoDB local hoặc cấu hình kết nối đến MongoDB Atlas.

Bước 5: Khởi động server bằng lệnh `npm run dev`.

Các lệnh quan trọng cho backend:

<code>npm install</code>	# Cài đặt dependencies
<code>npm run dev</code>	# Chạy ở chế độ development
<code>npm run build</code>	# Build production
<code>npm test</code>	# Chạy unit tests

A.1.3 Cài đặt Mobile App

Để cài đặt ứng dụng mobile, thực hiện các bước sau:

Bước 1: Đảm bảo Flutter SDK đã được cài đặt và PATH đã được cấu hình đúng.

Bước 2: Clone repository và di chuyển vào thư mục client/mobile.

Bước 3: Chạy lệnh flutter pub get để cài đặt dependencies.

Bước 4: Cấu hình file .env với API URL của backend.

Bước 5: Chạy ứng dụng bằng lệnh flutter run.

Để build file APK cho Android:

```
flutter pub get
flutter build apk --release
```

Để build file IPA cho iOS (cần máy Mac):

```
flutter pub get
flutter build ios --release
```

A.1.4 Cài đặt Web App

Để cài đặt ứng dụng web, thực hiện các bước sau:

Bước 1: Di chuyển vào thư mục client/web.

Bước 2: Chạy npm install để cài đặt dependencies.

Bước 3: Cấu hình file .env với API URL.

Bước 4: Chạy ứng dụng bằng lệnh npm run dev.

A.2 Hướng dẫn sử dụng

A.2.1 Đối với giáo viên

Giáo viên có thể thực hiện các thao tác sau:

Quản lý đề thi: Đăng nhập vào web dashboard, chọn mục “Quản lý đề thi”, nhấn “Tạo đề thi mới”, nhập thông tin và chọn câu hỏi từ ngân hàng, cấu hình số mã đề và nhấn “Xuất bản”.

Quản lý kết quả: Sau khi học sinh nộp bài, giáo viên có thể xem danh sách kết quả, chi tiết từng bài thi, và xuất báo cáo ra file Excel.

Phúc khảo: Khi có khiếu nại từ học sinh, giáo viên có thể xem lại hình ảnh phiếu đã quét và điều chỉnh điểm nếu cần.

A.2.2 Đối với học sinh

Học sinh có thể thực hiện các thao tác sau:

Quét phiếu: Mở ứng dụng mobile, chọn bài thi muốn nộp, đặt phiếu trả lời trong khung hình camera và nhấn chụp. Hệ thống sẽ tự động nhận diện và hiển thị kết quả.

Xem kết quả: Truy cập mục “Kết quả thi” để xem điểm số và chi tiết từng câu đã trả lời.

Khiếu nại: Nếu phát hiện sai sót, học sinh có thể gửi đơn khiếu nại trực tiếp từ ứng dụng.

A.3 Xử lý sự cố

A.3.1 Lỗi kết nối database

Nếu gặp lỗi kết nối MongoDB, kiểm tra: MongoDB service đã được khởi động chưa, URI kết nối trong file .env đã đúng chưa, và firewall không chặn port mặc định 27017.

A.3.2 Lỗi build Flutter

Nếu gặp lỗi khi build Flutter app, thử các bước sau: chạy flutter clean để xóa cache, chạy flutter pub get để cập nhật dependencies, và đảm bảo Flutter SDK và Dart SDK có phiên bản tương thích.

A.3.3 Lỗi CORS

Nếu gặp lỗi CORS khi gọi API từ web, kiểm tra cấu hình CORS trong backend đã cho phép origin của ứng dụng web chưa.